

oskernl2022-rv6

02_boot_trap — 启动/架构与 Trap/系统调用

Q02_001 · short_answer

启动入口在哪里？（例如 linker.ld 的 ENTRY、_start / start / head / entry 标签；必须给文件路径+符号证据）

项目	方法A	方法B
答案	启动入口位于 linker/kernel.ld:2 定义的 ENTRY(_entry)，实际汇编入口标签为 src/entry.S:5 的 _entry。链接脚本设置基址为 0x80200000，_entry 标签处开始执行多核启动检测逻辑。	linker/kernel.ld 中 ENTRY(_entry)，实际入口标签为 src/entry.S 中的 _entry
证据	[1] linker/kernel.ld · ENTRY OUTPUT_ARCH(riscv) ENTRY(_entry) BASE_ADDRESS = 0x80200000; [2] src/entry.S · _entry .globl _entry _entry: / check first boot here / la t0, __first_boot_magic ld t1, (t0) li t2, 0x5a5a bne t1, t2, __secondary_boot	[1] linker/kernel.ld · ENTRY OUTPUT_ARCH(riscv) ENTRY(_entry) [2] src/entry.S · _entry .section .text .extern __first_boot_magic .extern __get_boot_hartid .globl _entry _entry:

Q02_004 · short_answer

模式切换涉及的关键寄存器/位是什么？（例如 RISC-V mstatus/sstatus、x86 cr0/cr4/eflags；必须给证据摘录）

项目	方法A	方法B
答案	RISC-V S 模式切换涉及： 1. sstatus 寄存器的 SPP 位 (bit 8)：清除为 0 表示返回用户模式 2. sstatus 寄存器的 SPIE 位 (bit 5)：设置使用户模式中断 3. sepc 寄存器：设置返回用户态的 PC 4. satp 寄存器：切换用户页表 5. sret 指令：执行特权级切换返回 证据：src/trap.c:154-170 usertrapret() 函数中通过 w_sstatus/w_sepc/w_satp 设置后调用 sret。	sstatus 寄存器的 SPP 位 (bit 8) 和 SPIE 位 (bit 5)，以及 sepc、satp 寄存器
证据	[1] src/trap.c · usertrapret unsigned long x = r_sstatus(); x &= ~SSTATUS_SPP; // clear SPP to 0 for user mode x = SSTATUS_SPIE; // enable interrupts in user mode w_sstatus(x); w_sepc(p->trapframe->epc); uint64 satp = MAKE_SATP(p->pagetable); uint64 fn = TRAMPOLINE + (userret - trampoline); ((void (*)(uint64,uint64))fn)(TRAPFRAME, satp); [2] src/trampoline.S · userret csrw satp, a1 sfence.vma sret	[1] src/include/riscv.h · SSTATUS_SPP #define SSTATUS_SPP (1L << 8) // Previous mode, 1=Supervisor, 0=User #define SSTATUS_SPIE (1L << 5) // Supervisor Previous Interrupt Enable [2] src/trap.c · usertrapret w_sstatus(x); w_sepc(p->trapframe->epc); uint64 satp = MAKE_SATP(p->pagetable);

Q02_006 · short_answer

从入口汇编/固件交接到 C/Rust 主入口函数的跳转链是什么？（列出 3-6 个关键节点并给证据）

项目	方法A	方法B
答案	启动跳转链： 1. linker/kernel.ld:ENTRY(_entry) 设置入口点 2. src/entry.S:_entry 汇编入口，检测__first_boot_magic判断是否首核启动 3. src/entry.S:_secondary_boot 次级核启动路径，设置每核独立栈 4. src/main.c:main() C 语言内核主入口，执行 kvminit/trapinithart/procinit 等初始化 5. src/main.c:scheduler() 启动调度器运行第一个用户进程 证据：src/entry.S:5-22 调用 call main，src/main.c:44-95 完整初始化流程。	_entry (src/entry.S) → main (src/main.c) → scheduler (src/proc.c) → usertrap (src/trap.c) → 用户进程
证据	[1] src/entry.S · _entry _entry: la t0, __first_boot_magic ld t1, (t0) li t2, 0x5a5a bne t1, t2, _secondary_boot la sp, boot_stack_top call __get_boot_hartid _secondary_boot: mv t0, x0 add t0, a0, 1 slli t0, t0, 15 la sp, boot_stack add sp, sp, t0 call main [2] src/main.c · main void main(unsigned long hartid, unsigned long dtb_pa) { inithartid(hartid); if (__first_boot_magic == 0x5a5a) { __first_boot_magic = 0; cpuinit(); printfinit(); kpmunit(); kmallocinit(); kvminit(); kvminithart(); trapinithart(); procinit(); userinit(); started=1; } else { while (started == 0); kvminithart(); trapinithart(); } scheduler(); }	[1] src/entry.S · _entry _entry: ... call main [2] src/main.c · main void main(unsigned long hartid, unsigned long dtb_pa) { ... scheduler(); } [3] src/proc.c · scheduler void scheduler() { ... swtch(&c->context, &p->context); }

Q02_007 · fill_in

早期初始化 (Early Initialization) 各项状态（每项必须 implemented / stub / not_found + 证据路径，格式：项目：状态 [路径]）： - BSS 清零 (BSS Clearing): __ - 早期串口输出 (Early Serial/UART Output): __ - 设备树解析 (Device Tree Blob parsing, DTB): __ - 页表初始化时机 (Page Table Init): __（在 MMU 启用前/后？）

项目	方法A	方法B
答案	BSS 清零 (BSS Clearing): implemented [linker/kernel.ld:48-52 定义.sbss.bss.bss.*段，由链接器自动处理] 早期串口输出 (Early Serial/UART Output): implemented [src/printf.c:30-34 使用 sbi_console_putchar 输出] 设备树解析 (Device Tree Blob parsing, DTB): not_found [main() 接收 dtb_pa 参数但未发现解析代码] 页表初始化时机 (Page Table Init): implemented [src/main.c:63-64 kvminit 在 MMU 启用前建立映射, kvminithart 开启 MMU]	BSS 清零 (BSS Clearing): not_found [linker/kernel.ld 中有.bss 段定义但未见显式清零代码] 早期串口输出 (Early Serial/UART Output): implemented [src/printf.c 使用 sbi_console_putchar] 设备树解析 (Device Tree Blob parsing, DTB): not_found [main 接收 dtb_pa 参数但未见解析代码] 页表初始化时机 (Page Table Init): implemented [MMU 启用前, kvminit() 在 kvminithart() 之前调用]
证据	[1] linker/kernel.ld · bss_section .bss : { (.bss.stack) sbss_clear = .; (.sbss .bss .bss.*) ebss_clear = .; } [2] src/printf.c · putc if(c == '\b') { sbi_console_putchar('\b'); sbi_console_putchar(' '); sbi_console_putchar('\b'); } else { sbi_console_putchar(c); } [3] src/main.c · main kvminit(); // create kernel page table kvminithart(); // turn on paging	[1] linker/kernel.ld · .bss .bss : { (.bss.stack) sbss_clear = .; (.sbss .bss .bss.*) ebss_clear = .; } [2] src/printf.c · consputc void consputc(int c) { if(c == BACKSPACE){ sbi_console_putchar('\b'); ... } else { sbi_console_putchar(c); } } [3] src/main.c · main kvminit(); // create kernel page table kvminithart(); // turn on paging

Q02_010 · short_answer

构建系统如何选择目标平台/架构与入口文件？（Cargo features/Kconfig/Makefile 条件；必须引用配置证据）

项目	方法A	方法B
答案	Makefile 条件编译： 1. 平台选择：MAC?=SIFIVE_U，支持 QEMU 和 SIFIVE_U，通过-D\$(MAC) 传递宏定义 2. 文件系统选择：FS?=FAT，支持 FAT 和 RAM，通过-D\$(FS) 传递 3. 架构固定为 RISC-V 64：CFLAGS += -march=rv64g -mcmmodel=medany 4. 入口文件固定：\$K/entry.o 始终链接，由 linker/kernel.ld 的 ENTRY(_entry) 指定 证据：Makefile:6-15 平台宏定义，Makefile:73-77 编译标志。	通过 Makefile 中的 MAC 变量选择平台 (QEMU/SIFIVE_U)，通过 FS 变量选择文件系统 (FAT/SD/RAM)，使用-D 宏传递条件编译标志
证据	[1] Makefile · platform_config FS?=FAT MAC?=SIFIVE_U ifeq (\$(MAC),SIFIVE_U) DISK:=\$K/link_null.o endif ifeq (\$(MAC),QEMU) DISK:=\$K/link_disk.o endif CFLAGS += -MD -mcmmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax CFLAGS += -D\$(FS) -D\$(MAC) [2] Makefile · qemu_target QEMUOPTS = -machine \$(M) -bios \$(SBI) -kernel \$K/kernel -smp \$(CPUS) -nographic qemu: \$K/kernel \$(QEMU) \$(QEMUOPTS)	[1] Makefile · platform_select MAC?=SIFIVE_U FS?=FAT ifeq (\$(MAC),SIFIVE_U) DISK:=\$K/link_null.o endif ifeq (\$(MAC),QEMU) DISK:=\$K/link_disk.o endif CFLAGS = ... -D\$(FS) -D\$(MAC)

Q02_012 · tri_state_impl

MMU 启用前后是否存在串口/UART 地址切换逻辑（物理地址→虚拟地址）？（必须三态；搜索 phys_to_virt|virt_to_phys 及 UART 基址常量）

项目	方法A	方法B
答案	implemented	stub
证据	[1] src/include/memlayout.h · UART0_V #define UART0 0x10000000L // 256 MB #define UART0_V (UART0 + VIRT_OFFSET) #define VIRT_OFFSET 0x3F0000000L [2] src/vm.c · kvminit kvmmap(KERNBASE, KERNBASE, (uint64)etext - KERNBASE, PTE_R PTE_X); kvmmap((uint64)etext, (uint64)etext, PHYSTOP - (uint64)etext, PTE_R PTE_W);	[1] src/include/memlayout.h · UART0_V #define UART0 0x10000000L #define UART0_V (UART0 + VIRT_OFFSET) [2] src/printf.c · uart_access conputc 使用 sbi_console_putchar 而非直接访问 UART 寄存器，未见显式 phys_to_virt 切换逻辑。

Q02_014 · short_answer

是否支持多平台启动（StarFive VisionFive2/LoongArch/多板型）？（搜索 visionfive|jh7110|loongarch；有则描述差异入口与互斥关系；无则写未发现）

项目	方法A	方法B
答案	未发现多平台支持。代码仅支持 QEMU sifive_u 机和 FU740 板型（通过 MAC=SIFIVE_U 或 MAC=QEMU 切换）。搜索 visionfive、jh7110、loongarch 关键词均 0 命中。Makefile 中仅通过-D\$(MAC) 条件编译区分 QEMU 和 SIFIVE_U 两种平台，差异在于磁盘驱动链接对象不同（link_null.o vs link_disk.o）。	支持 QEMU 和 SiFive U740 (fu740) 双平台，通过 Makefile 中 MAC 变量切换。未见 visionfive、jh7110、loongarch 相关代码。
证据	[1] Makefile · platform_scan 搜索 visionfive jh7110 loongarch 均 0 命中。 Makefile 仅支持: MAC?=SIFIVE_U 或 MAC=QEMU [2] Makefile · disk_config ifeq (\$(MAC),SIFIVE_U) DISK:=\$K/link_null.o endif ifeq (\$(MAC),QEMU) DISK:=\$K/link_disk.o endif	[1] README.md · platform xv6 移植到 qemu 的 sifive_u 以及 fu740 的板子上 make all platform=qemu make all platform=sifive_u [2] Makefile · platform_search 在附件中检索 visionfive jh7110 loongarch, 未命中相关实现。

Q02_015 · short_answer

trap/异常向量入口在哪里？（trap_handler/trap_vector/__alltraps 等；必须给证据）

项目	方法A	方法B
答案	Trap 向量入口分两种： 1. 用户态 trap: src/trampoline.S:17 的 uservec 标签，通过 w_stvec 设置到 stvec 寄存器 2. 内核态 trap: src/kernelvec.S:9 的 kernelvec 标签，在 trapinithart() 中设置 证据: src/trap.c:54 w_stvec((uint64)kernelvec) 设置内核向量， src/trap.c:138 w_stvec(TRAMPOLINE + (uservec - trampoline)) 设置用户向量。	用户态 trap 入口：src/trampoline.S 中的 uservec；内核态 trap 入口：src/kernelvec.S 中的 kernelvec
证据	[1] src/trampoline.S · uservec .globl uservec uservec: csrrw a0, sscratch, a0 sd ra, 40(a0) sd sp, 48(a0) ... [2] src/kernelvec.S · kernelvec .globl kernelvec .align 4 kernelvec: addi sp, sp, -256 sd ra, 0(sp) ... [3] src/trap.c · trapinithart w_stvec((uint64)kernelvec);	[1] src/trampoline.S · uservec .globl uservec uservec: csrrw a0, sscratch, a0 sd ra, 40(a0) ... [2] src/kernelvec.S · kernelvec .globl kernelvec kernelvec: addi sp, sp, -256 sd ra, 0(sp) ...

Q02_017 · short_answer

TrapFrame/寄存器保存结构体定义在哪里？寄存器数量与字节数是多少？（必须引用结构体定义证据）

项目	方法A	方法B
答案	定义于 <code>src/include/trap.h:17-60</code> 的 <code>struct trapframe</code> 。 包含寄存器： - 内核元数据: <code>kernel_satp/kernel_sp/kernel_trap/epc/kernel_hartid</code> (5 个, 40 字节) - 通用寄存器: <code>ra/sp/gp/tp/t0-t2/s0-s1/a0-a7/s2-s11/t3-t6</code> (33 个, 264 字节) 总计 38 个字段, 288 字节 (0-280 行, 最后一个 t6 在偏移 280, 占 8 字节)。 证据: <code>src/include/trap.h:17-60</code> 完整定义, 注释标明每个字段偏移。	定义在 <code>src/include/trap.h</code> 中的 <code>struct trapframe</code> , 包含 32 个通用寄存器 + 5 个内核元数据字段, 共 37 个 <code>uint64</code> , 总计 296 字节
证据	[1] <code>src/include/trap.h · trapframe</code> <code>struct trapframe { / 0 / uint64 kernel_satp; / 8 / uint64 kernel_sp; / 16 / uint64 kernel_trap; / 24 / uint64 epc; / 32 / uint64 kernel_hartid; / 40 / uint64 ra; / 48 / uint64 sp; ... / 280 / uint64 t6; };</code>	[1] <code>src/include/trap.h · trapframe</code> <code>struct trapframe { / 0 / uint64 kernel_satp; / 8 / uint64 kernel_sp; / 16 / uint64 kernel_trap; / 24 / uint64 epc; / 32 / uint64 kernel_hartid; / 40 / uint64 ra; / 48 / uint64 sp; ... / 280 / uint64 t6; };</code>

Q02_020 · short_answer

选择一个具体 syscall (优先 `sys_write`) , 追踪: 用户指令 → trap → 分发 → 实现体。列出 3-6 个关键节点并给证据。

项目	方法A	方法B
答案	<code>sys_write</code> 调用链: 1. 用户态执行 <code>ecall</code> 指令触发 trap 2. <code>src/trampoline.S:uservec</code> 保存寄存器到 <code>trapframe</code> , 切换到内核页表, 跳转到 <code>usertrap</code> 3. <code>src/trap.c:usertrap:93</code> 检测 <code>scause==EXCP_ENV_CALL(8)</code> , 调用 <code>syscall()</code> 4. <code>syscall/syscall.c:syscall:8</code> 从 <code>a7</code> 读取 <code>syscall</code> 号, 从 <code>syscalls[]</code> 数组索引获取函数指针 5. <code>src/sysfile.c:sys_write:234</code> 执行实际写操作, 调用 <code>filewrite()</code> 6. <code>src/trap.c:usertrapret:133</code> 恢复用户态上下文, 通过 <code>trampoline.S:userret</code> 的 <code>sret</code> 返回 证据: <code>src/trap.c:93-105</code> <code>syscall</code> 分发, <code>src/sysfile.c:234-246</code> <code>sys_write</code> 实现。	<code>sys_write</code> 追踪链: 1.用户执行 <code>ecall (usrinit/initcode.S)</code> → 2. <code>uservec</code> 保存上下文 (<code>src/trampoline.S</code>) → 3. <code>usertrap</code> 识别 <code>syscall (src/trap.c)</code> → 4. <code>syscall()</code> 分发 (<code>syscall/syscall.c</code>) → 5. <code>sys_write</code> 实现 (<code>src/sysfile.c</code>) → 6. <code>filewrite</code> 执行写入 (<code>src/file.c</code>)
证据	[1] <code>src/trap.c · usertrap</code> <code>uint64 cause = r_scause(); if(cause == EXCP_ENV_CALL){ p->trapframe->epc += 4; intr_on(); syscall(); }</code> [2] <code>syscall/syscall.c · syscall</code> <code>num = p->trapframe->a7; if(num > 0 && num < NELEM(syscalls) && syscalls[num]) { p->trapframe->a0 = syscalls[num]; }</code> [3] <code>src/sysfile.c · sys_write</code> <code>uint64 sys_write(void) { int fd; uint64 addr; int n; if(argfd(0, &fd, NULL) < 0) return -1; if(argaddr(1, &addr) < 0) return -1; if(argint(2, &n) < 0) return -1; return filewrite(fd, addr, n); }</code>	[1] <code>syscall/sys.sh · entry</code> <code>entry 64 write</code> [2] <code>src/sysfile.c · sys_write</code> <code>uint64 sys_write(void) { int fd; struct file f; int n; uint64 p; if(argfd(0, &fd, &f) < 0 argint(2, &n) < 0 argaddr(1, &p) < 0) return -1; return filewrite(f, p, n); }</code> [3] <code>src/file.c · filewrite</code> <code>int filewrite(struct file f, uint64 addr, int n) { switch (f->type) { case FD_ENTRY: ewrite(f->ep, 1, addr, f->off, n); ... } }</code>

Q02_021 · short_answer

列出 5-10 个“高价值 syscall”（fork/exec/mmap/open/write 等）的实现三态（implemented/stub/not_found），并为每个至少给一条证据。

项目	方法A	方法B
答案	高价值 syscall 实现状态： 1. sys_execve: implemented [src/sysproc.c:11-34 完整实现，调用 exec()] 2. sys_write: implemented [src/sysfile.c:234-246 调用 filewrite()] 3. sys_read: implemented [src/sysfile.c:218-232 调用 fileread()] 4. sys_openat: implemented [src/sysfile.c:39-143 完整实现] 5. sys_mmap: implemented [src/sysfile.c:895-922 调用 do_mmap()] 6. sys_munmap: implemented [src/sysfile.c:924-932 调用 do_munmap()] 7. sys_clone: implemented [src/sysproc.c:109-124 调用 clone()] 8. sys_exit: implemented [src/sysproc.c:173-181 调用 exit()] 9. sys_kill: implemented [src/syssig.c:94-100 调用 kill()] 10. sys_brk: implemented [src/sysproc.c:163-171 调用 growproc()] 所有 syscall 均有实际逻辑实现，非桩函数。	fork: implemented [src/proc.c clone 函数] exec: implemented [src/sysproc.c sys_execve] open: implemented [src/sysfile.c sys_openat] write: implemented [src/sysfile.c sys_write] read: implemented [src/sysfile.c sys_read] exit: implemented [src/sysproc.c sys_exit] wait: implemented [src/sysproc.c sys_wait4] mmap: implemented [src/sysfile.c sys_mmap] munmap: implemented [src/sysfile.c sys_munmap] kill: implemented [src/syssig.c sys_kill]
证据	[1] src/sysproc.c · sys_execve <pre>uint64 sys_execve() { char path[FAT32_MAX_PATH], argv[MAXARG], env[MAXARG]; if(argstr(0, path, FAT32_MAX_PATH) < 0) return -1; if((argvlen = argstrvec(1,argv, MAXARG)) < 0) return -1; int ret = exec(path, argv, env); return ret; }</pre> [2] src/sysfile.c · sys_write <pre>uint64 sys_write(void) { int fd; uint64 addr; int n; if(argfd(0, &fd, NULL) < 0) return -1; if(argaddr(1, &addr) < 0) return -1; if(argint(2, &n) < 0) return -1; return filewrite(fd, addr, n); }</pre> [3] src/sysfile.c · sys_mmap <pre>uint64 sys_mmap(void) { uint64 start, len; int prot, flags, fd, off; if(argaddr(0, &start) < 0) return -1; ... uint64 ret = do_mmap(start, len, prot, flags, fd, off); return ret; }</pre>	[1] src/proc.c · clone <pre>int clone(uint64 flag, uint64 stack, uint64 ptid, uint64 tls, uint64 ctid) { struct proc np; if((np = allocproc(p, 0)) == NULL) return -1; ...</pre> [2] src/sysproc.c · sys_execve <pre>uint64 sys_execve() { char path[FAT32_MAX_PATH], argv[MAXARG], *env[MAXARG]; ... exec(path, argv, env); }</pre> [3] src/sysfile.c · sys_mmap <pre>uint64 sys_mmap(void) { uint64 start, len; int prot, flags, fd, off; ... return do_mmap(start, len, prot, flags, fd, off); }</pre>

Q02_025 · short_answer

缺页异常与内存特性（CoW/lazy）是否在 trap 中联动？（若存在，说明入口点与调用到内存模块的证据）

项目	方法A	方法B
答案	声明但未发现完整实现。src/include/vm.h:42-43 声明了 handle_page_fault 和 kernel_handle_page_fault 函数，但在代码中搜索 handle_page_fault 仅找到声明，未发现实际实现和调用点。README 声称“完成了缺页中断的处理”，但代码中 usertrap() 的异常处理分支仅处理 EXCP_ENV_CALL(系统调用) 和 devintr(设备中断)，对缺页异常 (EXCP_LOAD_PAGE=13/EXCP_STORE_PAGE=15) 仅打印错误并设置 p->killed=SIGTERM，未调用页故障处理函数。证据：src/trap.c:120-126 缺页时仅打印错误。	存在缺页异常处理框架，但未见明确的 CoW (Copy-on-Write) 实现。页表项中有 PTE_D (Dirty) 位用于 munmap 时写回判断，但 trap 中未见缺页异常到内存模块的明确联动代码。

项目	方法A	方法B
证据	[1] src/include/vm.h · handle_page_fault int handle_page_fault(int kind, uint stval); int kernel_handle_page_fault(int kind, uint stval); [2] src/trap.c · page_fault_scan 搜索 handle_page_fault 实现: 0 命中 (仅头文件声明)。usertrap() 中对缺页异常的处理: else { printf("\nusertrap(): unexpected scause %p pid=%d %s\n", r_scause(), p->pid, p->name); p->killed = SIGTERM; } [3] README.md · claims - 完成了缺页中断的处理	[1] src/trap.c · page_fault 在附件中检索 handle_page_fault、 page_fault 等关键字, src/vm.h 中声明了 int handle_page_fault(int kind, uint stval) 但未见完 整实现。 [2] src/mmap.c · do_munmap if(pte & PTE_D) { // write back when PTE_D bit is 1 pa = PTE2PA(pte); fwrite(f, va, size); }

Q02_026 · short_answer

与 09 多核交叉一致性: per-CPU trap 栈/时钟初始化顺序与 AP 上线是否一致? (互指证据或写单核不适用)

项目	方法A	方法B
答案	多核实现一致。src/main.c:44-95 显示: 1. 首核 (hart 0): 执行 kvmininit()->kvminithart()->trapinithart()->procinit(), 然后启动其他核 2. 次级核 (hart 1-4): 等待 started==0 自旋, 然后执行 kvminithart()->trapinithart() 每核独立调用 trapinithart() 设置 stvec, 符合 per-CPU trap 初始化要求。时钟初始化 timerinit() 仅在首核执行一次 (全局 tickslock), 但 set_next_timeout() 在每核 trapinithart() 中调用, 确保每核都有独立的定时器中断。证据: src/main.c:63-68 首核初始化序列, src/main.c:83-88 次级核初始化。	一致。主核 (hart0) 先完成 kvmininit、trapinithart、procinit 等初始化, 然后通过 start_hart 唤醒其他核, 其他核等待 started 标志后执行 kvminithart 和 trapinithart。
证据	[1] src/main.c · main if (__first_boot_magic == 0x5a5a) { kvmininit(); kvminithart(); trapinithart(); for(int i = 1; i < NCPU; i++) { if(hartid!=i&&booted[i]==0){ start_hart(i, (uint64)_entry, 0); } } } else { while (started == 0); kvminithart(); trapinithart(); }	[1] src/main.c · main if (__first_boot_magic == 0x5a5a) { kvmininit(); trapinithart(); procinit(); for(int i = 1; i < NCPU; i++) { if(hartid!=i&&booted[i]==0) { start_hart(i, (uint64)_entry, 0); } } started=1; } else { while (started == 0); kvminithart(); trapinithart(); }

Q02_027 · fill_in

Syscall 实现全量统计 (Syscall Coverage Analysis), 请按格式填写: - 分发表路径: __ - 完整实现  (implemented): __ 个 - 桩/ENOSYS/return 0  (stub): __ 个, 代表性例子: __ - 未注册  (not_found): __ 个 - 统计依据 (grep 或 outline 方式): __ (若无法精确计数, 给出区间估计并说明理由)

项目	方法A	方法B
答案	分发表路径: syscall/syscall.c:8-17 (引用 syscalls[] 数组) 完整实现  (implemented): 40+ 个 (基于 src/sysproc.c、src/sysfile.c、src/systime.c、src/syssig.c、src/sypoll.c 中的 sys_ 函数统计) 桩/ENOSYS/return 0  (stub): 2 个, 代表性例子: sys_exit_group [src/syssig.c:18-21 直接 return 0]、sys_ppoll [src/sypoll.c:14-16 直接 return 0] 未注册  (not_found): 0 个 (所有 syscall 均通过 sys.sh 脚本生成到 syscalls[] 数组) 统计依据: grep_in_repo 搜索 'sys_[a-z0-9_](' 命中 53 个函数定义, 检查 syscall/syscall.c 的 syscall() 函数通过 syscalls[num] 间接调用, sys.sh 脚本生成分发表。文档 doc/内核实现--系统调用.md:374 列出 syscalls[] 数组结构。	分发表路径: syscall/syscall.c 完整实现  (implemented): 约 45-50 个 桩/ENOSYS/return 0  (stub): 约 5-10 个, 代表性例子: sys_ppoll (返回 0) 未注册  (not_found): 约 0-5 个 统计依据 (grep 或 outline 方式): syscall/sys.sh 生成约 55 个 syscall 入口, syscall/syscall.c 中 syscalls 数组填充, src/sysfile.c、src/sysproc.c、src/syssig.c、src/systime.c、src/syslog.c、src/sypoll.c 中查找实现
证据	[1] syscall/syscall.c · syscall <pre>if(num > 0 && num < NELEM(syscalls) && syscalls[num]) { p->trapframe->a0 = syscalls[num]; }</pre> [2] src/syssig.c · sys_exit_group <pre>uint64 sys_exit_group(void){ return 0; }</pre> [3] src/sypoll.c · sys_ppoll <pre>uint64 sys_ppoll(){ return 0; }</pre>	[1] syscall/sys.sh · entry_count syscall/sys.sh 中 entry 调用约 55 次, 生成 SYS_ 定义和 sys_ 函数声明 [2] src/sypoll.c · sys_ppoll <pre>uint64 sys_ppoll() { return 0; }</pre> [3] syscall/syscall.c · syscall <pre>if(num > 0 && num < NELEM(syscalls) && syscalls[num]) { p->trapframe->a0 = syscalls[num]; } else { printf("pid %d %s: unknown sys call %d\n", ...); p->trapframe->a0 = -1; }</pre>

Q02_028 · short_answer

README 与 syscall 声称对照: README 中声称兼容/实现了哪些 syscall 或标准? 与代码分发表实际是否一致? (无 README 则写「无 README, 仅以代码为准」)

项目	方法A	方法B
答案	README.md 未明确列出 syscall 兼容性声称, 仅在工作总结中提到: - "完善了用户内存管理和内核内存管理" - "完善了 mmap 的机制" - "完成了缺页中断的处理" - "完成了信号相关的操作" - "完成了对本地回环地址的 Socket 支持" 代码验证: - mmap: 已实现 sys_mmap/sys_munmap [src/sysfile.c:895-932] - 信号: 已实现 sys_kill/sys_rt_sigaction/sys_rt_sigprocmask/sys_rt_sigreturn [src/syssig.c] - 缺页中断: 仅声明 handle_page_fault [src/include/vm.h:42-43], 未发现实现和调用 - Socket: 未发现 socket 相关 syscall 实现 (搜索 socket 仅找到头文件声明) 结论: README 声称部分与代码一致 (mmap、信号), 缺页中断和 Socket 声称与代码不符。	README.md 未明确列出具体的 syscall 清单, 仅提及「完善了进程队列」「完善了用户内存管理和内核内存管理」「完成了 mmap 的机制」「完成了缺页中断的处理」「完成了信号相关的操作」「完成了轮询相关的操作」等功能模块。代码分发表实际实现了约 55 个 syscall, 与 README 功能描述基本一致。

项目	方法A	方法B
证据	[1] README.md · claims - 完善了 <code>mmap</code> 的机制 - 完成了缺页中断的处理 - 完成了信号相关的操作 [2] src/sysfile.c · sys_mmap<pre>uint64 sys_mmap(void) { uint64 start, len; int prot, flags, fd, off; ... uint64 ret = do_mmap(start, len, prot, flags, fd, off); return ret; }</pre>	[1] README.md · work ## 我们的工作 - 完成了进程队列相关的工作 - 完善了用户内存管理和内核内存管理, 实现了块分配和页分配以及 <code>vma</code> 的操作 - 完善了 <code>mmap</code> 的机制 - 完成了缺页中断的处理 - 完成了信号相关的操作 - 完成了轮询相关的操作

Q02_029 · short_answer

`_impl` 命名模式搜索结论: `grep _impl\b|sys_[a-z0-9_]*_impl`, 结果是命中了哪些函数 (列出), 还是「未见该命名模式」? (必须给搜索结论)

项目	方法A	方法B
答案	未见该命名模式。在 repos/oskernrl2022-rv6 全仓库搜索 <code>_impl\b sys_[a-z0-9_]*_impl</code> 模式, 0 命中。该仓库采用直接命名方式 (如 <code>sys_write</code> 、 <code>sys_execve</code>), 未使用 <code>_impl</code> 后缀分离接口与实现的命名模式。syscall 分发直接通过 <code>syscalls[]</code> 数组索引调用 <code>sys_*</code> 函数。	未见该命名模式。在附件源码中 <code>grep _impl\b sys_[a-z0-9_]*_impl</code> 未命中任何函数。系统调用实现使用 <code>sys_</code> 命名 (如 <code>sys_write</code> 、 <code>sys_read</code>), 无 <code>_impl</code> 后缀。
证据	[1] repos/oskernrl2022-rv6 · impl_pattern_scan<pre>grep_in_repo 搜索 'impl\b sys[a-z0-9_]*_impl': 0 命中 (搜索 145 个文件)。 仓库采用直接命名: <code>sys_write</code>、<code>sys_execve</code> 等, 无 <code>_impl</code> 后缀。</pre>	[1] src/sysfile.c · impl_search<pre>在附件中检索 <code>impl\b sys[a-z0-9_]*_impl</code> 模式, 未命中相关实现。系统调用使用 <code>sys_write</code>、<code>sys_read</code> 等命名, 无 <code>_impl</code> 后缀。</pre>

Q02_032 · short_answer

信号发送支持哪些粒度? (搜索 `sys_kill/sys_tkill/sys_tgkill`; 分别是进程级/线程级/进程组级; 列出已实现的与其证据)

项目	方法A	方法B
答案	已实现: 1. 进程级: <code>sys_kill</code> [src/syssig.c:94-100] 通过 <code>kill(pid, sig)</code> 向指定 pid 进程发送信号 2. 线程组级: <code>sys_tgkill</code> [src/syssig.c:102-110] 通过 <code>tgkill(pid, tid, sig)</code> 向指定线程组内的线程发送信号 未发现: <code>sys_tkill</code> (纯线程级信号发送) <code>kill()</code> 实现 [src/proc.c:752-768] 遍历 <code>procs</code> 数组查找 pid, 设置 <code>p->sig_pending</code> 和 <code>p->killed</code>。 <code>tgkill()</code> 实现 [src/proc.c:785-791] 先通过 <code>cmp_parent()</code> 验证 pid-tid 的父子关系, 再调用 <code>kill(tid, sig)</code>。 证据: <code>src/syssig.c:94-110</code> <code>sys_kill/sys_tgkill</code> 定义, <code>src/proc.c:752-791</code> <code>kill/tgkill</code> 实现。	支持进程级 (<code>sys_kill</code>) 和线程级 (<code>sys_tgkill</code>)。 <code>sys_kill</code> 按 pid 发送信号给进程, <code>sys_tgkill</code> 按 pid+tid 发送信号给特定线程。未见 <code>sys_tgkill</code> 进程组级实现。

项目	方法A	方法B
证据	[1] src/syssig.c · sys_kill uint64 sys_kill(){ int sig, pid; argint(0,&pid); argint(1,&sig); return kill(pid,sig); } [2] src/syssig.c · sys_tgkill uint64 sys_tgkill(){ int sig, tid, pid; argint(0,&pid); argint(1,&tid); argint(2,&sig); return tgkill(pid,tid,sig); } [3] src/proc.c · kill int kill(int pid,int sig){ for(p = proc; p < &proc[NPROC]; p++){ if(p->pid == pid){ p->sig_pending.__val[0] = 1ul << sig; p->killed = sig; } }	[1] src/syssig.c · sys_kill uint64 sys_kill() { int sig, pid; argint(0, &pid); argint(1, &sig); return kill(pid, sig); } [2] src/syssig.c · sys_tgkill uint64 sys_tgkill() { int sig, tid, pid; argint(0, &pid); argint(1, &tid); argint(2, &sig); return tgkill(pid, tid, sig); } [3] src/proc.c · tgkill int tgkill(int pid, int tid, int sig) { if(!cmp_parent(pid, tid)) return -1; else return kill(tid, sig); }

03_mem_mgmt — 内存管理（物理/虚拟/分配器）

Q03_004 · short_answer

物理页帧分配器的核心数据结构是什么？（例如 bitmap 数组、buddy free list、slab cache 表、struct run 单链表等；必须引用结构体/字段证据）

项目	方法A	方法B
答案	struct run 单链表 + 全局 kmem 结构体。核心结构：struct run { struct run next; } 作为空闲链表节点；static struct { struct spinlock lock; struct run freelist; uint64 npage; } kmem 管理全局空闲链表和页计数	struct run 单链表：struct run { struct run *next; }; 配合全局 kmem 结构体包含 freelist 指针和 spinlock 锁。
证据	[1] src/pm.c · run struct run { struct run next; }; [2] src/pm.c · kmem static struct { struct spinlock lock; struct run freelist; uint64 npage; } kmem;	[1] src/pm.c · run struct run { struct run next; }; static struct { struct spinlock lock; struct run freelist; uint64 npage; } kmem;

Q03_005 · short_answer

物理分配器的并发控制锁粒度是什么？（全局大锁 / per-CPU / 分桶 / 无锁+关中断 / 其他；必须给锁对象类型与持锁范围证据）

项目	方法A	方法B
答案	全局自旋锁 (global spinlock)。使用 struct spinlock kmem.lock 保护整个空闲链表，持锁范围覆盖 allocpage 和 freepage 的整个临界区	全局大锁：kmem.lock (struct spinlock)，持锁范围覆盖整个 allocpage/freepage 操作。

项目	方法A	方法B
证据	<pre>[1] src/pm.c · allocpage acquire(&kmem.lock); r = kmem.freelist; ... release(&kmem.lock); [2] src/pm.c · freepage acquire(&kmem.lock); r->next = kmem.freelist; kmem.freelist = r; ... release(&kmem.lock); [3] src/include/spinlock.h · spinlock struct spinlock { uint locked; char name; struct cpu cpu; };</pre>	<pre>[1] src/pm.c · allocpage acquire(&kmem.lock); r = kmem.freelist; if(r) kmem.freelist = r->next; release(&kmem.lock); 全局锁保 护整个空闲链表。 [2] src/pm.c · kmem static struct { struct spinlock lock; struct run *freelist; uint64 npage; } kmem; 单一锁保护所有物理页分 配。</pre>

Q03_007 · short_answer

页表操作 API (walk/map/unmap 或等价) 对应的函数名/模块是什么？列出 1-3 个关键入口并给证据。

项目	方法A	方法B
答案	核心 API: walk() 遍历页表返回 PTE 指针; mappages() 建立虚拟地址到物理地址的映射; vmunmap() 解除映射并可选释放物理页。辅助 API: uvmalloc() 分配用户内存并建立映射; uvmdealloc() 释放用户内存	关键入口: 1) walk() - 页表遍历获取 PTE; 2) mappages() - 建立虚拟到物理映射; 3) vmunmap() - 解除映射。均在 src/vm.c 模块。
证据	<pre>[1] src/vm.c · walk pte_t *walk(pagetable_t pagetable, uint64 va, int alloc) [2] src/vm.c · mappages int mappages(pagetable_t pagetable, uint64 va, uint64 size, uint64 pa, int perm) [3] src/vm.c · vmunmap void vmunmap(pagetable_t pagetable, uint64 va, uint64 npages, int do_free)</pre>	<pre>[1] src/vm.c · walk pte_t *walk(pagetable_t pagetable, uint64 va, int alloc) - 根据虚拟地址找到对应页表项 PTE。 [2] src/vm.c · mappages int mappages(pagetable_t pagetable, uint64 va, uint64 size, uint64 pa, int perm) - 完成虚拟地址到物理地址的映射。 [3] src/vm.c · vmunmap void vmunmap(pagetable_t pagetable, uint64 va, uint64 npages, int do_free) - 释放虚拟地址空间映射。</pre>

Q03_008 · short_answer

页表修改路径的并发控制是什么？（锁粒度、是否需要关中断、是否使用每进程地址空间锁等；必须引用锁/临界区证据）

项目	方法A	方法B
答案	页表修改路径本身无专用锁，依赖物理页分配时的 kmem.lock 全局锁保护。mappages/walk 在分配新页表页时调用 allocpage() 持有 kmem.lock。无 per-CPU 锁或地址空间锁，无显式关中断保护	每进程地址空间锁：通过 proc->lock 保护进程页表修改，无专门页表锁。内核页表在启动时一次性建立后只读。
证据	<pre>[1] src/vm.c · walk if(!alloc (pagetable = (pde_t*)allocpage()) == NULL) return NULL; - 调用 allocpage 间接持有 kmem.lock [2] src/pm.c · allocpage acquire(&kmem.lock); ... release(&kmem.lock);</pre>	<pre>[1] src/proc.c · proc struct proc { struct spinlock lock; pagetable_t pagetable; ... }; 进程锁保护包括页 表在内的进程状态。 [2] src/vm.c · uvmalloc uint64 uvmalloc(pagetable_t pagetable, uint64 start, uint64 end, int perm) - 页表修 改在进程上下文中进行，由调用者持有 proc->lock。</pre>

Q03_010 · tri_state_impl

是否存在缺页异常 (Page Fault) 处理逻辑并与内存分配/映射联动？（必须三态）

项目	方法A	方法B
答案	not_found	stub
证据	[1] src/trap.c · EXCP_LOAD_PAGE #define EXCP_LOAD_PAGE 0xd // 13 - 定义但未在 usertrap 中处理 [2] src/trap.c · usertrap usertrap 中仅处理 EXCP_ENV_CALL 和 devintr, EXCP_LOAD_PAGE/EXCP_STORE_PAGE 分支被注释掉 [3] src/include/vm.h · handle_page_fault int handle_page_fault(int kind, uint stval); - 仅声明，未发现实现	[1] src/trap.c · usertrap usertrap() 中处理 scause, 但 EXCP_LOAD_PAGE (0xd) 和 EXCP_STORE_PAGE (0xf) 未明确处理, 仅打印错误并设置 p->killed。 [2] src/vm.h · handle_page_fault int handle_page_fault(int kind, uint stval); 声明存在但源码中未见完整实现联动分配/映射。 [3] src/trap.c · exception_handling else { printf("usertrap(): unexpected scause %p", r_scause()); p->killed = SIGTERM; } - 缺页异常未实现完整处理链路。

Q03_011 · short_answer

追踪一条缺页链路：trap/异常入口 → 缺页处理函数（handle_page_fault 或等价）→ 分配页帧 → 建立映射。用 3-5 个关键节点描述并给每节点证据。

项目	方法A	方法B
答案	缺页链路未实现。trap.c:usertrap 中 EXCP_LOAD_PAGE/EXCP_STORE_PAGE 处理分支被注释掉, handle_page_fault 仅在 vm.h 中声明但无实现。候选链路（未闭合）：usertrap [trap.c:94] → handle_excp [trap.c:未实现] → handle_page_fault [vm.h:42 仅声明] → allocpage [pm.c:79]	候选链路（未完全闭合）：1) usertrap [src/trap.c:usertrap] → 2) 缺页 scause 检测 [src/trap.c:usertrap] → 3) handle_page_fault [src/vm.h:声明] → 4) allocpage [src/pm.c:allocpage] → 5) mappages [src/vm.c:mappages]。注：handle_page_fault 实现不完整，链路未完全闭合。
证据	[1] src/trap.c · usertrap / else if(handle_excp(cause) == 0) { } / - 缺页处理被注释 [2] src/include/vm.h · handle_page_fault int handle_page_fault(int kind, uint stval); - 仅声明	[1] src/trap.c · usertrap usertrap() 捕获异常, scause 判断中断/异常类型, 但缺页异常 (0xd/0xf) 未明确处理。 [2] src/vm.h · handle_page_fault int handle_page_fault(int kind, uint stval); 声明存在但实现不完整。 [3] src/pm.c · allocpage void *allocpage(void) - 分配物理页帧。 [4] src/vm.c · mappages int mappages(pagetable_t pagetable, uint64 va, uint64 size, uint64 pa, int perm) - 建立页表映射。

Q03_016 · tri_state_impl

是否存在 Page Cache（页缓存/文件页缓存）管理？（必须三态）

项目	方法A	方法B
答案	not_found	stub
证据	[1] src/bio.c · bio.c 块 I/O 缓冲, 但非页缓存机制	[1] src/bio.c · buf struct buf { int valid; uint sectorno; uchar data[BSIZE]; ... }; - 块缓存存在, 但为磁盘块级别 (512 字节), 非页缓存 (4KB)。 [2] src/bio.c · bget static struct buf* bget(uint dev, uint sectorno) - LRU 块缓存管理, 但非页缓存。 [3] src/ · page_cache_search 搜索 page_cache/radix_tree/address_space 等页缓存相关结构, 未发现。

Q03_017 · tri_state_impl

是否存在脏页回写 (dirty page writeback) 机制? (必须三态; 若 implemented 需指出同步/异步与触发条件)

项目	方法A	方法B
答案	implemented	stub
证据	[1] src/mmap.c · do_munmap if(pte & PTE_D){ pa = PTE2PA(pte); filewrite(f, va, size); } - munmap 时同步回写脏页	[1] src/mmap.c · do_munmap if(pte & PTE_D){ pa = PTE2PA(pte); filewrite(f, va, size); } - munmap 时检查脏位并写回, 但非后台异步写回。 [2] src/ · writeback_search 搜索 background_writeback/kswapd/cleaner_thread 等后台写回机制, 未发现。

Q03_019 · short_answer

TLB 刷新指令/函数点是什么? (RISC-V sfence.vma / AArch64 tlbi / x86 invlpg 等, 或仓库中等价的 TLB 刷新封装; 必须给证据)

项目	方法A	方法B
答案	RISC-V sfence.vma 指令。封装函数: sfence_vma() 定义在 riscv.h:329, 使用 asm volatile("sfence.vma") 实现。调用点: vm.c:57 kvminithart 中调用 sfence_vma()	RISC-V sfence.vma 指令, 封装为 sfence_vma() 函数。
证据	[1] src/include/riscv.h · sfence_vma static inline void sfence_vma() { asm volatile("sfence.vma"); } [2] src/vm.c · kvminithart w_satp(MAKE_SATP(kernel_pagetable)); sfence_vma();	[1] src/include/riscv.h · sfence_vma static inline void sfence_vma() { asm volatile("sfence.vma"); } - RISC-V TLB 刷新指令封装。 [2] src/vm.c · kvminithart void kvminithart() { w_satp(MAKE_SATP(kernel_pagetable)); sfence_vma(); } - 页表切换后刷新 TLB。

Q03_020 · short_answer

用户指针安全检查机制是什么? (access_ok/verify_area/UserInPtr 等; 列出入口点与校验逻辑证据)

项目	方法A	方法B
答案	walkaddr() 函数进行用户指针检查。检查逻辑：1) va >= MAXVA 返回 NULL；2) walk(pagetable, va, 0) 检查 PTE 存在；3) (pte & PTE_V) == 0 检查有效位；4) (pte & PTE_U) == 0 检查用户可访问位。copyin/copyout 调用 walkaddr 进行校验	copyin/copyout/copyinstr 函数通过 walkaddr() 验证用户虚拟地址有效性，检查 PTE_U 位确保用户可访问。
证据	<pre>[1] src/vm.c · walkaddr if(va >= MAXVA) return NULL; pte = walk(...); if((pte & PTE_V) == 0) return NULL; if((pte & PTE_U) == 0) return NULL; [2] src/copy.c · copyin pa0 = walkaddr(pagetable, va0); if(pa0 == NULL) return -1; [3] src/copy.c · copyout pa0 = walkaddr(pagetable, va0); if(pa0 == NULL) return -1;</pre>	<pre>[1] src/copy.c · copyin int copyin(pagetable_t pagetable, char dst, uint64 srcva, uint64 len) { pa0 = walkaddr(pagetable, va0); if(pa0 == NULL) return -1; ... } - 通过 walkaddr 验证用户地址。 [2] src/vm.c · walkaddr uint64 walkaddr(pagetable_t pagetable, uint64 va) { if((pte & PTE_U) == 0) return NULL; ... } - 检查 PTE_U 位确保用户可访问。 [3] src/copy.c · copyinstr int copyinstr(pagetable_t pagetable, char *dst, uint64 srcva, uint64 max) - 安全复制用户字符串，逐页验 证。</pre>

Q03_023 · fill_in

物理内存总量 (Physical Memory Size) : KB/MB ; 页大小 (Page Size) : bytes; 最大进程虚拟地址空间 (Virtual Address Space) : bits。 (必须从代码常量/链接脚本/配置中给出证据; 无法确定则写 unknown 并说明已搜路径)

项目	方法A	方法B
答案	物理内存总量: 128 MB; 页大小: 4096 bytes; 最大进程虚拟地址空间: 39 bits (Sv39)	物理内存总量: 128 MB; 页大小: 4096 bytes; 最大进程虚拟地址空间: 39 bits (Sv39, 256GB MAXVA)。
证据	<pre>[1] src/include/memlayout.h · PHYSTOP #define PHYSTOP (0x80000000ULL + (unsigned long long)(1ULL * 128 * 1024 * 1024)) // 128MB [2] src/include/riscv.h · PGSIZE #define PGSIZE 4096 // bytes per page [3] src/include/riscv.h · MAXVA #define MAXVA (1L << (9 + 9 + 9 + 12 - 1)) // 256 GB, Sv39 39-bit virtual address [4] src/include/riscv.h · SATP_SV39 #define SATP_SV39 (8L << 60) // use riscv's sv39 page table scheme</pre>	<pre>[1] src/include/memlayout.h · PHYSTOP #define PHYSTOP (0x80000000ULL + (unsigned long long) (1ULL * 128 * 1024 * 1024)) // 128MB - 物理内存上限。 [2] src/include/riscv.h · PGSIZE #define PGSIZE 4096 // bytes per page - 页大小 4KB。 [3] src/include/riscv.h · MAXVA #define MAXVA (1L << (9 + 9 + 9 + 12 - 1)) // 256 GB - Sv39 三级页表, 39 位虚拟地址。 [4] src/include/riscv.h · SATP_SV39 #define SATP_SV39 (8L << 60) - 使用 Sv39 页表模式。</pre>

Q03_024 · single_choice

内存保护机制 (Memory Protection) 的实现形式更接近哪种? (Stallings Ch7.1)

项目	方法A	方法B
答案	C. 硬件页表 + 软件指针检查双重保护	A. 纯硬件页表权限位 (R/W/X/U 位, MMU 负责拒绝非法访问)

项目	方法A	方法B
证据	[1] src/include/riscv.h · PTE_U #define PTE_U (1L << 4) // 1 -> user can access - 硬件页表权限位 [2] src/copy.c · copyin walkaddr 检查 PTE_U 位, 软件校验用户指针 [3] src/vm.c · walkaddr if((*pte & PTE_U) == 0) return NULL; - 软件检查用户可访问性	[1] src/include/riscv.h · PTE_flags #define PTE_V (1L << 0) #define PTE_R (1L << 1) #define PTE_W (1L << 2) #define PTE_X (1L << 3) #define PTE_U (1L << 4) - 硬件页表权限位。 [2] src/vm.c · walkaddr if((pte & PTE_U) == 0) return NULL; - MMU 通过 PTE_U 位拒绝内核访问用户页。 [3] src/vm.c · mappages pte = PA2PTE(pa) perm PTE_V PTE_A PTE_D; - 设置页表权限位由硬件 MMU 执行保护。

Q03_025 · short_answer

逻辑内存组织 (Logical Memory Organization, Stallings Ch7.1): 进程地址空间中 text/data/heap/stack/mmap 各区域 (或等价区间) 是否由统一的映射管理结构 (VMA/区间表/链表/BTreeMap 等) 维护? (如存在请给结构体证据; 不存在则写未发现等价结构)

项目	方法A	方法B
答案	是, 使用 VMA (Virtual Memory Area) 双向链表统一管理。结构体 struct vma 定义在 vma.h:15, 包含 type (LOAD/HEAP/STACK/MMAP/TRAP 等)、addr、sz、end、perm、fd、f_off 字段。进程 struct proc 包含 vma 头指针	是, 由 struct vma 双向链表维护。vma 结构包含 type 字段区分 LOAD/HEAP/STACK/MMAP/TRAP 等区域类型。
证据	[1] src/include/vma.h · vma struct vma { enum segtype type; int perm; uint64 addr; uint64 sz; uint64 end; int flags; int fd; uint64 f_off; struct vma prev; struct vma next; }; [2] src/include/proc.h · proc struct proc { ... struct vma *vma; ... }; [3] src/vma.c · vma_list_init 初始化 LOAD/HEAP/STACK/MMAP/TRAP 各区域 VMA	[1] src/include/vma.h · vma struct vma { enum segtype type; int perm; uint64 addr; uint64 sz; uint64 end; ... struct vma prev; struct vma next; }; - VMA 双向链表管理内存区域。 [2] src/include/vma.h · segtype enum segtype {NONE, LOAD, TEXT, DATA, BSS, HEAP, MMAP, STACK, TRAP}; - 区域类型枚举。 [3] src/proc.h · proc struct proc { ... struct vma *vma; ... }; - 每个进程有 vma 链表头。

Q03_027 · single_choice

取页策略 (Fetch Policy, Stallings Ch8.2) 更接近哪种?

项目	方法A	方法B
答案	D. 预分配 (Pre-allocation): 进程创建时立即分配全部物理页	A. 按需调页 (Demand Paging): 缺页时才分配物理页
证据	[1] src/vma.c · alloc_vma alloc==1 时立即调用 uvmmalloc 分配物理页并建立映射 [2] src/vma.c · alloc_mmap_vma 调用 alloc_vma(p, MMAP, addr, sz, perm, 1, NULL) - 立即分配	[1] src/vma.c · alloc_vma alloc_vma(..., int alloc, ...) - alloc 参数控制是否立即分配, 但多数调用 alloc=1 立即分配。 [2] src/exec.c · loadelf loadelf() 中 alloc_load_vma() 分配 VMA 并立即映射物理页, exec 时预分配。 [3] src/trap.c · demand_paging_search 缺页处理未完全实现, 但 VMA 机制支持按需分配框架。

Q03_028 · short_answer

放置策略 (Placement Policy, Stallings Ch8.2)：新的匿名映射或堆区域增长时，系统如何选择虚拟地址区间？（固定起始地址 / mmap_base 向下生长 / 首次适配 / 最佳适配 等；必须给实现证据或写未发现等价策略）

项目	方法A	方法B
答案	VMA 双向链表首次适配 (first-fit) 。alloc_vma 遍历链表查找空闲区间：while(nvma != vma_head) { if(end <= nvma->addr) break; ... }。MMAP 区域从 USER_MMAP_START 向下生长 (alloc_mmap_vma 中 addr = PGROUNDDOWN(mvma->addr - sz))	堆区域：从 LOAD 段末尾向上增长 (alloc_addr_heap_vma)。mmap 区域：从 USER_MMAP_START 向下分配 (alloc_mmap_vma)。栈：固定 USER_STACK_BOTTOM 向下。
证据	[1] src/vma.c · alloc_vma 遍历 VMA 链表查找空闲区间：while(nvma != vma_head) { if(end <= nvma->addr) break; } [2] src/vma.c · alloc_mmap_vma addr = PGROUNDDOWN(mvma->addr - sz); - MMAP 向下生长 [3] src/include/memlayout.h · USER_MMAP_START #define USER_MMAP_START (USER_STACK_BOTTOM - 0x10000000)	[1] src/vma.c · alloc_addr_heap_vma struct vma alloc_addr_heap_vma(struct proc p, uint64 addr, int perm) { struct vma *lvma = type_locate_vma(p->vma, LOAD); uint64 start = lvma->end; ... } - 堆从 LOAD 末尾开始。 [2] src/vma.c · alloc_mmap_vma if(addr == 0) { addr = PGROUNDDOWN(mvma->addr - sz); } - mmap 从 MMAP 区域顶部向下分配。 [3] src/include/memlayout.h · USER_MMAP_START #define USER_MMAP_START (USER_STACK_BOTTOM - 0x10000000) - mmap 区域起始地址定义。

Q03_030 · short_answer

内存主链路（必须给出，尽量以 Mermaid graph TD 表达）：从确认的最强内存入口（缺页处理入口/mmap 入口/brk 入口/等价入口）出发，追踪到页表操作核心点或物理页分配核心点，写出 3-6 个关键节点。节点格式：FuncName [path:line]。若链路未被源码证据完全闭合，标注候选主链路而非确认的主链路。只画一条主链，不要并列展开多条支线。

项目	方法A	方法B
答案	graph TD sys_mmap[src/sysfile.c:894] --> do_mmap[do_mmap [src/mmap.c:30]] do_mmap --> alloc_mmap_vma[alloc_mmap_vma [src/vma.c:195]] alloc_mmap_vma --> alloc_vma[alloc_vma [src/vma.c:64]] alloc_vma --> uvmmalloc[uvmmalloc [src/vm.c:224]] uvmmalloc --> mappages[mappages [src/vm.c:85]] mappages --> walk[walk [src/vm.c:140]] walk --> allocpage[allocpage [src/pm.c:79]]	候选主链路 (mmap 入口到页表操作)：\n\nusertrap [src/trap.c:157] --> syscall [src/trap.c:177] --> sys_mmap [src/sysfile.c:724] --> do_mmap [src/mmap.c:58] --> alloc_mmap_vma [src/vma.c:147] --> alloc_vma [src/vma.c:53] --> mappages [src/vm.c:125] --> walk [src/vm.c:107]\n\n注：缺页处理链路未完全闭合 (handle_page_fault 实现不完整)，此为 mmap 系统调用链路。

项目	方法A	方法B
证据	[1] <code>src/sysfile.c · sys_mmap</code> 系统调用入口 [2] <code>src/mmap.c · do_mmap</code> mmap 核心实现 [3] <code>src/vma.c · alloc_mmap_vma</code> 分配 MMAP VMA [4] <code>src/vma.c · alloc_vma</code> 通用 VMA 分配 [5] <code>src/vm.c · uvmmalloc</code> 用户内存分配 [6] <code>src/vm.c · mappages</code> 页表映射 [7] <code>src/pm.c · allocpage</code> 物理页分配	[1] <code>src/trap.c · usertrap</code> usertrap() 捕获系统调用，调用 <code>syscall()</code> 分发。 [2] <code>src/sysfile.c · sys_mmap</code> uint64 sys_mmap(void) { ... return do_mmap(start, len, prot, flags, fd, off); } [3] <code>src/mmap.c · do_mmap</code> uint64 do_mmap(...) { struct vma vma = alloc_mmap_vma(p, flags, start, len, perm, fd, offset); ... } [4] <code>src/vma.c · alloc_mmap_vma</code> struct vma alloc_mmap_vma(...) { vma = alloc_vma(p, MMAP, addr, sz, perm, 1, NULL); ... } [5] <code>src/vm.c · mappages</code> int mappages(pagetable_t pagetable, uint64 va, uint64 size, uint64 pa, int perm) { if((pte = walk(pagetable, a, 1)) == NULL) return -1; ... }

Q03_032 · single_choice

地址重定位 (Address Relocation, Stallings Ch7.1) 的绑定时机更接近哪种？

项目	方法A	方法B
答案	C. 运行时动态绑定 (Run-time / Dynamic Relocation): 通过 MMU 基址 + 界限或页表在每次访问时转换	C. 运行时动态绑定 (Run-time / Dynamic Relocation): 通过 MMU 基址+界限或页表在每次访问时转换
证据	[1] <code>src/include/riscv.h · SATP_SV39</code> RISC-V Sv39 页表机制，运行时通过 <code>satp</code> 寄存器切换页表 [2] <code>src/vm.c · kvmminihart</code> <code>w_satp(MAKE_SATP(kernel_pagetable));</code> <code>sfence_vma();</code> - 运行时切换页表	[1] <code>src/include/riscv.h · w_satp</code> static inline void w_satp(uint64 x) { asm volatile("csrw satp, %0" : : "r" (x)); } - 运行时设置页表基址寄存器。 [2] <code>src/vm.c · kvmminihart</code> void kvmminihart() { w_satp(MAKE_SATP(kernel_pagetable)); sfence_vma(); } - 每核运行时加载页表。 [3] <code>src/trampoline.S · userret</code> <code>csrw satp, a1</code> - 用户返回时切换页表，每次内存访问通过 MMU 转换。

04_process_smp — 进程/线程/调度与多核

Q04_001 · short_answer

执行实体 (Execution Entity) 抽象是什么？请按以下格式作答（每项必须有代码证据）： - 顶层类型名: (如 *Process / Task / Thread / TaskControlBlock*) - 结构体路径: __ - 关键字段 (至少列 3 个): *Context=*, *State=*, *PID=*, *TrapFrame=* - 是否区分 PCB 与 TCB: (是 / 否 / 待核实)

项目	方法A	方法B
答案	顶层类型名: proc 结构体路径: src/include/proc.h:128 关键字段: Context=context (struct context), State=state (enum procstate), PID=pid (int), TrapFrame=trapframe (struct trapframe*) 是否区分 PCB 与 TCB: 否	顶层类型名: proc/Task 结构体路径: src/include/proc.h 关键字段: Context=context (struct context), State=state (enum procstate), PID=pid (int), TrapFrame=trapframe (struct trapframe*) 是否区分 PCB 与 TCB: 否
证据	[1] src/include/proc.h · proc struct proc { int magic; struct spinlock lock; enum procstate state; struct proc parent; void chan; int killed; int xstate; int pid; int uid; int gid; uint64 kstack; uint64 sz; pagetable_t pagetable; struct trapframe *trapframe; struct context context; ... }	[1] src/include/proc.h · proc struct proc { ... enum procstate state; ... int pid; ... struct trapframe *trapframe; ... struct context context; ... } [2] src/include/cpu.h · context struct context { uint64 ra; uint64 sp; uint64 s0-s11; };

Q04_002 · short_answer

任务/进程的生命周期状态机有哪些状态与流转点？（Ready/Running/Blocked/Exited 等；需状态枚举/字段证据）

项目	方法A	方法B
答案	状态枚举定义于 src/include/proc.h:88: enum procstate { UNUSED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE }。流转点: UNUSED→RUNNABLE (allocproc/clone/fork), RUNNABLE→RUNNING (scheduler), RUNNING→SLEEPING (sleep), SLEEPING→RUNNABLE (wakeup), RUNNING→ZOMBIE (exit), ZOMBIE→UNUSED (freeproc 被 wait4pid 回收)	状态枚举: UNUSED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE 流转点: - UNUSED->RUNNABLE: allocproc()/userinit() - RUNNABLE->RUNNING: scheduler() - RUNNING->SLEEPING: sleep() - SLEEPING->RUNNABLE: wakeup() - RUNNING->ZOMBIE: exit() - ZOMBIE->UNUSED: freeproc() via wait4pid()
证据	[1] src/include/proc.h · procstate enum procstate { UNUSED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE }; [2] src/proc.c · scheduler p->state = RUNNING; swtch(&c->context, &p->context); [3] src/proc.c · sleep p->state = SLEEPING; sched(); [4] src/proc.c · exit p->state = ZOMBIE; sched();	[1] src/include/proc.h · procstate enum procstate { UNUSED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE }; [2] src/proc.c · scheduler p->state = RUNNING; ... swtch(&c->context, &p->context); [3] src/proc.c · sleep p->state = SLEEPING; sched(); [4] src/proc.c · exit p->state = ZOMBIE; sched(); [5] src/proc.c · wakeup p->state = RUNNABLE; readyq_push(p);

Q04_004 · short_answer

上下文切换保存/恢复了哪些寄存器集合？（例如 RISC-V s0-s11；必须引用汇编/结构体证据）

项目	方法A	方法B
答案	保存/恢复的寄存器：ra, sp, s0-s11（共 14 个寄存器）。证据：src/swtch.S:5-28 显示 sd ra/sp/s0-s11 到 context 结构，ld 从 context 恢复。struct context 定义于 src/include/cpu.h:9-26 包含 ra, sp, s0-s11 字段	保存/恢复的寄存器：ra, sp, s0, s1, s2, s3, s4, s5, s6, s7, s8, s9, s10, s11（共 14 个寄存器） 证据：src/swtch.S 中 swtch 函数保存 callee-saved 寄存器
证据	[1] src/swtch.S · swtch sd ra, 0(a0); sd sp, 8(a0); sd s0, 16(a0); sd s1, 24(a0); sd s2, 32(a0); sd s3, 40(a0); sd s4, 48(a0); sd s5, 56(a0); sd s6, 64(a0); sd s7, 72(a0); sd s8, 80(a0); sd s9, 88(a0); sd s10, 96(a0); sd s11, 104(a0) [2] src/include/cpu.h · context struct context { uint64 ra; uint64 sp; uint64 s0; uint64 s1; ... uint64 s11; };	[1] src/swtch.S · swtch sd ra, 0(a0) sd sp, 8(a0) sd s0, 16(a0) sd s1, 24(a0) sd s2, 32(a0) sd s3, 40(a0) sd s4, 48(a0) sd s5, 56(a0) sd s6, 64(a0) sd s7, 72(a0) sd s8, 80(a0) sd s9, 88(a0) sd s10, 96(a0) sd s11, 104(a0) [2] src/include/cpu.h · context struct context { uint64 ra; uint64 sp; uint64 s0; uint64 s1; ... uint64 s11; };

Q04_005 · short_answer

调度算法 (Scheduling Algorithm) 属于哪类？请按格式作答：- 算法名称: **(必须是以下之一: FCFS / Round-Robin (RR) / Stride/Proportional-Share / MLFQ / CFS / Priority / 其他)** - 代码证据 (关键字段/函数) : __ - RR: timeslice/slice 字段位置= - Stride: stride 字段与比较逻辑位置= - MLFQ: 多级队列 VecDeque/数组层级证据= - Priority: priority 字段参与 pick_next 排序证据=__

项目	方法A	方法B
答案	算法名称: Round-Robin (RR) 代码证据 (关键字段/函数) : 全局就绪队列 readyq (src/proc.c:29), scheduler() 通过 readyq_pop() 按 FIFO 顺序取进程 (src/proc.c:124), 配合时钟中断 proc_tick() 触发 yield() 实现时间片轮转。无 priority/stride/timeslice 字段, 为简单 RR	算法名称: Round-Robin (RR) 代码证据 (关键字段/函数) : - 简单就绪队列 FIFO 调度, 无优先级/时间片字段 - scheduler() 从 readyq 按顺序取进程 - 时钟中断触发 yield() 实现轮转
证据	[1] src/proc.c · readyq queue readyq; [2] src/proc.c · scheduler struct proc* p = readyq_pop(); if(p){ p->state = RUNNING; swtch(&c->context, &p->context); } [3] src/proc.c · yield readyq_push(p); p->state = RUNNABLE; sched();	[1] src/proc.c · scheduler struct proc* p = readyq_pop(); ... p->state = RUNNING; ... swtch(&c->context, &p->context); [2] src/proc.c · yield readyq_push(p); p->state = RUNNABLE; sched(); [3] src/trap.c · usertrap if(which_dev == 2) yield(); [4] src/include/proc.h · priority/timeslice 在 src/include/proc.h 中检索 priority/timeslice/slice 字段, 未发现相关字段

Q04_006 · short_answer

调度器 (Scheduler)核心入口/关键函数有哪些？（schedule/pick_next 等；给 1-3 个入口与证据）

项目	方法A	方法B
答案	核心入口：scheduler() (src/proc.c:119) 为每核死循环调度主入口； sched() (src/proc.c:520) 为进程主动让出 CPU 的底层切换函数； yield() (src/proc.c:657) 为进程主动让出并重新入队	核心入口： 1. scheduler() - 主调度循环 (src/proc.c) 2. yield() - 主动让出 CPU (src/proc.c) 3. sched() - 底层调度切换 (src/proc.c)
证据	[1] src/proc.c · scheduler void scheduler(){ while(1){ struct proc* p = readyq_pop(); ... swtch(&c->context, &p->context); } } [2] src/proc.c · sched void sched(void){ swtch(&p->context, &mycpu()->context); } [3] src/proc.c · yield void yield(){ acquire(&p->lock); readyq_push(p); p->state = RUNNABLE; sched(); release(&p->lock); }	[1] src/proc.c · scheduler void scheduler(){ struct cpu c = mycpu(); ... while(1){ struct proc p = readyq_pop(); ... swtch(&c->context, &p->context); } } [2] src/proc.c · yield void yield(){ struct proc *p = myproc(); acquire(&p->lock); readyq_push(p); p->state = RUNNABLE; sched(); release(&p->lock); } [3] src/proc.c · sched void sched(){ ... swtch(&p->context, &mycpu()->context); }

Q04_008 · short_answer

fork/clone 是否复制地址空间与文件表？（必须给复制路径证据；若 stub 需说明形态）

项目	方法A	方法B
答案	是。地址空间复制：proc_pagetable() (src/proc.c:308) 调用 vma_copy() 和 vma_deep_mapping() 复制父进程 VMA 并深拷贝页表 (src/proc.c:345-352)。文件表复制：clone() 中循环调用 filedup() 复制 ofile (src/proc.c:453-455)，edup() 复制 cwd (src/proc.c:456)	地址空间：是，通过 vma_deep_mapping() 深拷贝或 vma_shallow_mapping() 浅拷贝 文件表：是，通过 filedup() 复制打开文件引用 证据路径：src/proc.c clone() 函数
证据	[1] src/proc.c · proc_pagetable if(thread_create) { vma_shallow_mapping(...) } else { vma_deep_mapping(...) } [2] src/proc.c · clone for(i = 0; i < NOFILE; i++) if(p->ofile[i]) np->ofile[i] = filedup(p->ofile[i]); np->cwd = edup(p->cwd);	[1] src/proc.c · clone for(i = 0; i < NOFILE; i++) if(p->ofile[i]) np->ofile[i] = filedup(p->ofile[i]); np->cwd = edup(p->cwd); [2] src/proc.c · proc_pagetable if(thread_create) { ... vma_shallow_mapping(pp->pagetable, p->pagetable, nvma) ... } else { ... vma_deep_mapping(pp->pagetable, p->pagetable, nvma) ... } [3] src/vma.c · vma_deep_mapping memmove(mem, (char *)pa, PGSIZE); if(mappages(new, start, PGSIZE, (uint64)mem, flags) != 0)...

Q04_012 · short_answer

PID 分配器实现是什么？（自增/bitmap/空闲栈复用/只分配不回收；必须给证据）

项目	方法A	方法B
答案	简单自增计数器，无回收复用。证据：src/proc.c:35 定义 int nextpid = 1; src/proc.c:158-160 allocpid() 中 acquire(&pid_lock); pid = nextpid; nextpid = nextpid + 1; release(&pid_lock); 无 free_pid 或复用逻辑	实现方式：自增计数器 (nextpid)，带锁保护 证据：src/proc.c allocpid() 函数使用 pid_lock 保护 nextpid 自增
证据	[1] src/proc.c · nextpid int nextpid = 1; [2] src/proc.c · allocpid int allocpid() { acquire(&pid_lock); pid = nextpid; nextpid = nextpid + 1; release(&pid_lock); return pid; }	[1] src/proc.c · allocpid int allocpid() { int pid; acquire(&pid_lock); pid = nextpid; nextpid = nextpid + 1; release(&pid_lock); return pid; } [2] src/proc.c · nextpid int nextpid = 1; struct spinlock pid_lock;

Q04_013 · short_answer

父子进程树如何存储？（children Vec/链表/parent+sibling 指针；必须给结构体字段证据）

项目	方法A	方法B
答案	单向 parent 指针。struct proc 包含 struct proc *parent 字段 (src/include/proc.h:133)。findchild() 遍历全局 proc 数组查找 parent==p 的子进程 (src/proc.c:612-626)。无 children 链表或 sibling 指针	存储方式：parent 指针 + 全局 proc 数组遍历 结构体字段：struct proc { struct proc *parent; ... } 查找子进程：findchild() 遍历 proc 数组检查 np->parent == p
证据	[1] src/include/proc.h · proc struct proc parent; // Parent process [2] src/proc.c · findchild for(struct proc np = proc; np < &proc[NPROC]; np++){ if(np->parent == p && cond(np, pid)){ ... } }	[1] src/include/proc.h · proc struct proc { ... struct proc parent; ... int pid; ... } [2] src/proc.c · findchild struct proc findchild(struct proc p,int (cond) (struct proc,int),int pid,struct proc child){ for(struct proc np = proc; np < &proc[NPROC]; np++){ if(np->parent == p&&cond(np,pid)){ ... } } [3] src/proc.c · reparent for(pp = proc; pp < &proc[NPROC]; pp++){ if(pp-> parent == p){ acquire(&pp->lock); pp->parent = initproc; release(&pp->lock); } }

Q04_014 · tri_state_impl

是否实现信号 (signal) 或 futex？（若二者都无则 not_found；若只实现其一需说明并给证据）

项目	方法A	方法B
答案	stub	implemented

项目	方法A	方法B
证据	[1] src/signal.c · sighandle <pre>void sighandle(void) { if(p->killed){ signum = p->killed; ... frame = allocpage(); tf = allocpage(); p- >trapframe = tf; ... } }</pre> [2] src/include/proc.h · do_futex <pre>int do_futex(int uaddr, int futex_op, int val, ktime_t timeout, int *addr2, int val2, int val3); // 仅声明, 未发现实现</pre> [3] src/include/proc.h · FUTEX_WAIT <pre>#define FUTEX_WAIT 0 ... #define FUTEX_WAKE 1 // 常量已定义但无实现函数</pre>	[1] src/signal.c · signal handling <pre>int set_sigaction(...); int sigprocmask(...); void sighandle(void); void sigreturn(void);</pre> [2] src/include/signal.h · signal structures <pre>struct sigaction { union { __sighandler_t sa_handler; } __sigaction_handler; __sigset_t sa_mask; int sa_flags; }; struct sig_frame { __sigset_t mask; struct trapframe tf; struct sig_frame next; };</pre> [3] doc/内核实现--Futex.md · futex documentation Futex 系统调用: futex(uint32_t *uaddr, int futex_op, uint32_t val, ...); 操作类型: FUTEX_WAIT, FUTEX_WAKE, FUTEX_REQUEUE

Q04_015 · short_answer

与 09 多核的交叉一致性：是否存在每核队列/任务迁移/IPI resched？（需与第 9 章互指证据或写不适用）

项目	方法A	方法B
答案	全局单就绪队列 readyq，无每核队列。scheduler() 每核从同一全局 readyq 取任务 (src/proc.c:124)，无锁保护 readyq 操作存在竞争。未发现任务迁移/负载均衡逻辑。IPI 仅用于启动 AP (start_hart)，无 IPI resched 机制	每核运行队列：否，使用全局 readyq 和 waitq_pool 任务迁移：未发现实现 IPI resched：未发现实现 证据：src/proc.c 中 readyq 为全局变量，scheduler() 每核独立运行但共享队列
证据	[1] src/proc.c · readyq <pre>queue readyq; // 全局单队列</pre> [2] src/proc.c · scheduler <pre>struct proc* p = readyq_pop(); // 每核共享</pre> [3] src/include/sbi.h · send_ipi <pre>static inline void send_ipi(uint64 mask) { a_sbi_ecall(0x735049, 0, mask, 0, 0, 0, 0, 0); } // 仅用于 启动</pre>	[1] src/proc.c · readyq <pre>queue readyq; ... void readyq_push(struct proc p){ queue_push(&readyq,p); } struct proc readyq_pop(){ return queue_pop(&readyq); }</pre> [2] src/proc.c · scheduler <pre>void scheduler(){ struct cpu c = mycpu(); ... while(1){ struct proc p = readyq_pop(); ... } }</pre> [3] src/proc.c · load_balance/migrate 在 src/ 中检索 load_balance/migrate/IPI resched，未发现相关实现

Q04_016 · short_answer

exit() 资源回收路径：调用链是什么？是否真正回收地址空间/文件表/通知父进程？（必须给调用链证据；桩则说明）

项目	方法A	方法B
答案	调用链：exit() (src/proc.c:721) → 关闭文件 (fileclose) → eput(cwd) → wakeup(parent) → reparent() → p->state=ZOMBIE → sched()。真正回收：freeproc() (src/proc.c:167) 被 wait4pid 调用，释放 ofile/kstack/pagetable/vma。通知父进程：wakeup(getparent(p)) (src/proc.c:745)	调用链：sys_exit() -> exit() -> freeproc() 回收内容： - 文件表：关闭所有打开文件 (fileclose) - 地址空间：freeproc() 调用 proc_freepagetable() 释放页表 - 通知父进程：wakeup(getparent(p)) - 状态：设置为 ZOMBIE 等待 wait4pid 回收

项目	方法A	方法B
证据	<pre>[1] src/proc.c · exit for(int fd=0; fdofile[fd]=0; } eput(p->cwd); wakeup(getparent(p)); reparent(p); p->state=ZOMBIE; sched(); [2] src/proc.c · freeproc if(p->ofile) kfree((void)p->ofile); if(p->kstack) freepage((void)p->kstack); if(p->pagetable) proc_freepagetable(p);</pre>	<pre>[1] src/proc.c · exit void exit(int n) { ... for(int fd = 0; fd < NOFILE; fd++){ if(p->ofile[fd]){ fclose(f); p->ofile[fd] = 0; } } ... wakeup(getparent(p)); reparent(p); p->state = ZOMBIE; sched(); } [2] src/proc.c · freeproc static void freeproc(struct proc p) { ... if(p->pagetable) proc_freepagetable(p); p- >pagetable = 0; ... p->state = UNUSED; } [3] src/vm.c · uvmfree void uvmfree(struct proc p) { free_vma_list(p); freewalk(p->pagetable); }</pre>

Q04_021 · single_choice

用户线程与内核线程的映射模型 (User-Level Thread to Kernel-Level Thread Mapping) 更接近哪种？ (Stallings Ch4)

项目	方法A	方法B
答案	1:1 (每个用户线程对应一个内核线程, 如 Linux pthread)	D. 仅内核线程 (无独立用户线程库)
证据	<pre>[1] src/proc.c · clone clone() 通过 CLONE_THREAD CLONE_VM 创建共享地址空间的线程, 每个线程有独立 struct proc 和 trapframe, 1:1 映射</pre>	<pre>[1] src/include/proc.h · proc 仅 struct proc 表示内核级执行实体, 无用户态线程库结 构 [2] usrinit/user.h · user API fork() 通过 clone(0,0,0,0,0) 实现, 无独立用户线程 创建 API [3] src/proc.c · user_thread/pthread 在源码中检索 user_thread/pthread/thread_create 用户态库, 未发现独立用户线程库实现</pre>

Q04_022 · tri_state_impl

是否实现线程局部存储 (Thread-Local Storage, TLS)? (必须三态; 搜索 thread_local|TLS|_thread|#[thread_local]; 若 implemented 需说明 TLS 的访问方式: tp 寄存器/段寄存器/其他)

项目	方法A	方法B
答案	implemented	stub
证据	<pre>[1] src/proc.c · clone np->trapframe->tp = tls; // CLONE_SETTLS 时 设置 tp 寄存器 [2] src/include/proc.h · CLONE_SETTLS #define CLONE_SETTLS 0x00080000 / create a new TLS for the child / [3] src/cpu.c · cpuid int cpuid() { int id = r_tp(); return id; } // tp 寄存器用于 per-CPU 访问</pre>	<pre>[1] src/proc.c · clone (np->trapframe) = (p->trapframe); np->trapframe->tp = tls; ... if(flag & CLONE_SETTLS){ ... } [2] src/include/proc.h · CLONE_SETTLS #define CLONE_SETTLS 0x00080000 [3] src/ · thread_local/TLS 在 src/ 中检索 thread_local/#[thread_local]/__thread, 未发现完 整 TLS 访问机制实现 (tp 寄存器仅用于 hartid)</pre>

Q04_023 · multi_choice

调度器是否追踪/优化以下哪些性能指标 (Scheduling Criteria, Stallings Ch9)? (多选; 未发现则留空并在 notes 写 not_found)

项目	方法A	方法B
答案	["未发现调度性能统计"]	["F. 未发现调度性能统计"]
证据	[1] src/proc.c · scheduling_metrics_scan 搜索 CPU 利用率/吞吐量/周转时间/等待时间/响应时间相关统计变量或计算逻辑, 0 命中。scheduler() 为简单 FIFO 轮转, 无性能指标追踪	[1] src/include/proc.h · tms <pre>struct tms { uint64 utime; uint64 stime; uint64 cutime; uint64 cstime; };</pre> 仅用于时间统计, 非调度优化 [2] src/proc.c · throughput/turnaround/waiting/response 在 src/ 中检索 throughput/turnaround/waiting_time/response_time 等调度性能指标, 未发现追踪或优化代码 [3] src/timer.c · proc_tick <pre>void proc_tick(void) { ... p->proc_tms.utime += 1; ... }</pre> 仅累计时间, 不用于调度决策

Q04_030 · tri_state_impl

是否实现 IPI (核间中断) 发送与处理? (必须三态)

项目	方法A	方法B
答案	implemented	stub
证据	[1] src/include/sbi.h · send_ipi <pre>static inline void send_ipi(uint64 mask) { a_sbi_ecall(0x735049, 0, mask, 0, 0, 0, 0, 0); }</pre> [2] src/include/sbi.h · SBI_SEND_IPI <pre>#define SBI_SEND_IPI 4 // SBI 扩展定义</pre>	[1] src/sbi.h · send_ipi <pre>static inline void send_ipi(uint64 mask) { a_sbi_ecall(0x735049, 0, mask, 0, 0, 0, 0, 0); }</pre> [2] src/ · IPI handler 在 src/ 中检索 IPI 处理函数/核间中断处理例程, 未发现完整的 IPI 接收处理实现 [3] src/trap.c · devintr <pre>devintr()</pre> 处理外部中断但未明确处理 IPI 中断

Q04_031 · short_answer

若存在 IPI: 发送与处理路径分别在哪些函数/文件? (给关键入口与证据)

项目	方法A	方法B
答案	发送路径: send_ipi() (src/include/sbi.h:88) 通过 SBI ecall 0x735049 发送。处理路径: 未发现专用 IPI 处理函数, IPI 仅用于启动 AP, 无运行时 IPI 处理逻辑 (如 resched/TLB shutdown)	发送路径: src/sbi.h send_ipi() 通过 SBI 调用 处理路径: 未发现完整实现 (devintr() 未明确处理 IPI) 证据: IPI 发送通过 SBI HSM 扩展, 但接收处理逻辑不完整

项目	方法A	方法B
证据	<pre>[1] src/include/sbi.h · send_ipi static inline void send_ipi(uint64 mask) { a_sbi_ecall(0x735049, 0, mask, 0, 0, 0, 0, 0); } [2] src/ · ipi_handler_scan 搜索 ipi_handler/clear_ipi/process_ipi 等处理函数, 0 命中</pre>	<pre>[1] src/sbi.h · send_ipi static inline void send_ipi(uint64 mask) { a_sbi_ecall(0x735049, 0, mask,0,0,0,0,0); } [2] src/trap.c · devintr int devintr(void) { ... if ((0x8000000000000000L & scause) && 9 == (scause & 0xff)) { ... } } // 外部中断处理但 未明确 IPI</pre>

Q04_033 · short_answer

per-CPU 的实现方式是什么？（例如 TLS/tp 寄存器/gsbase/数组索引 hartid；需证据）

项目	方法A	方法B
答案	tp 寄存器 + 数组索引。cpuid() 通过 r_tp() 读取 tp 寄存器获取 hartid (src/cpu.c:25), mycpu() 返回 &cpus[id] (src/cpu.c:33)。启动时 inithartid() 设置 tp (src/main.c:17)	实现方式: tp 寄存器存储 hartid + 数组索引 - tp 寄存器: r_tp() 读取当前 hartid - 数组索引: cpus[cpuid()] 访问对应 CPU 结构 证据: src/cpu.c mycpu() 和 src/include/riscv.h r_tp()
证据	<pre>[1] src/cpu.c · cpuid int cpuid() { int id = r_tp(); return id; } [2] src/main.c · inithartid static inline void inithartid(unsigned long hartid) { asm volatile("mv tp, %0" :: "r" (hartid)); }</pre>	<pre>[1] src/include/riscv.h · r_tp static inline uint64 r_tp() { uint64 x; asm volatile("mv %0, tp" : "=r" (x)); return x; } [2] src/cpu.c · cpuid int cpuid() { int id = r_tp(); return id; } [3] src/cpu.c · mycpu struct cpu mycpu(void) { int id = cpuid(); struct cpu c = &cpus[id]; return c; } [4] src/main.c · inithartid static inline void inithartid(unsigned long hartid) { asm volatile("mv tp, %0" : : "r" (hartid)); }</pre>

Q04_036 · short_answer

与 03/04/05/08 章的交叉一致性 (Cross-Chapter Consistency), 按以下四项分别作答 (每项须给证据路径或写「单核不适用」): - 03 TLB: 多核页表修改后 TLB 刷新策略= - **04 调度: 每核运行队列/负载均衡/IPI resched**= - 05 Trap: per-CPU trap 栈/时钟中断初始化与 AP 上线顺序= - **08 锁: SpinLock 关中断行为在多核下是否安全=**

项目	方法A	方法B
答案	03 TLB: 多核页表修改后 TLB 刷新策略=未发现跨核 TLB 刷新, 仅 scheduler() 切换页表时 sfence_vma() (src/proc.c:137) 04 调度: 每核运行队列/负载均衡/IPI resched=全局单 readyq, 无每核队列/负载均衡/IPI resched 05 Trap: per-CPU trap 栈/时钟中断初始化与 AP 上线顺序=AP 上线前 trapinithart() 已调用 (src/main.c:79) 08 锁: SpinLock 关中断行为在多核下是否安全=acquire() 调用 push_off() 关中断 (src/spinlock.c:25), 多核下安全 (仅关本核中断)	03 TLB: 多核页表修改后 TLB 刷新策略=每核自行 sfence_vma(), 无跨核协调 04 调度: 每核运行队列/负载均衡/IPI resched=全局 readyq, 无负载均衡, 无 IPI resched 05 Trap: per-CPU trap 栈/时钟中断初始化与 AP 上线顺序=每核 trapinithart() 独立初始化, BSP 先初始化后唤醒 AP 08 锁: SpinLock 关中断行为在多核下是否安全=安全, acquire() 调用 push_off() 关中断防止同核重入

项目	方法A	方法B
证据	[1] src/proc.c · scheduler w_satp(MAKE_SATP(p->pagetable)); sfence_vma(); [2] src/spinlock.c · acquire push_off(); // disable interrupts to avoid deadlock. [3] src/main.c · main AP 路径: trapinithart(); scheduler();	[1] src/vm.c · sfence_vma sfence_vma() 在每核页表切换时调用, 无跨核同步 [2] src/proc.c · scheduler 每核独立运行 scheduler() 从全局 readyq 取任务 [3] src/main.c · main BSP 先完成初始化 (started=1), AP 等待 started 后执行 trapinithart() [4] src/spinlock.c · acquire void acquire(struct spinlock *lk) { push_off(); ... } // 关中断保护

Q04_038 · short_answer

NCPU/MAXCPU (或等价宏) 与链接脚本中的每 hart 栈/入口布局是否对应? (搜索 _max_hart_id 等; 给宏定义与链接脚本对应证据, 或写未发现)

项目	方法A	方法B
答案	NCPU=5 (src/include/param.h:4)。linker/kernel.ld 无 per-hart 栈布局, 仅定义内核基址 0x80200000 和段布局。cpus[NCPU] 在 .bss 段 (src/cpu.c:13)。未发现 _max_hart_id 或 per-hart 栈符号	NCPU=5 (src/include/param.h) 栈布局: src/entry.S 中 boot_stack 分配 409658 字节 (5 harts) 链接脚本: linker/kernel.ld 未显式指定每 hart 栈, 由 entry.S 汇编分配 对应关系: 一致, entry.S 中 .space 4096 * 5 * 8 对应 NCPU=5
证据	[1] src/include/param.h · NCPU #define NCPU 5 // maximum number of CPUs [2] linker/kernel.ld · kernel.ld SECTIONS { . = 0x80200000; .text : { (.text .text.) }bss : { (.bss.stack) (.sbss .bss .bss.*) } }	[1] src/include/param.h · NCPU #define NCPU 5 // maximum number of CPUs [2] src/entry.S · boot_stack .section .bss.stack .align 12 .globl boot_stack boot_stack: .space 4096 * 5 * 8 / 5 harts / [3] linker/kernel.ld · SECTIONS .bss : { *(.bss.stack) ... } // .bss.stack 段包含 boot_stack

05_fs_drivers — 文件系统与设备 I/O

Q05_001 · short_answer

VFS 抽象层 (Virtual File System, VFS)接口是什么形态? (Rust trait / C op 表; 必须给接口定义证据)

项目	方法A	方法B
答案	C 语言函数指针表 + 统一结构体。无独立 VFS 抽象层，fat32.c 直接实现文件系统接口。struct file 统一管理 FD_ENTRY/FD_PIPE/FD_DEVICE 三种类型，通过 type 字段区分。文件操作通过 fileinput/fileoutput 函数根据 type 分发到 eread/ewrite (FAT32)、piperead/pipewrite (pipe)、devsw[].read/write (设备)。证据： src/include/file.h:14-28 定义 struct file 含 type 枚举； src/file.c:145-175 定义 fileinput/fileoutput 分发逻辑； src/fat32.c 实现 eread/ewrite/dirlookup 等具体 FS 操作。	C 操作表/函数指针表 (C operation table / function pointer table) 。在 src/fat32.c 中 struct fs 定义包含 disk_init/disk_read/disk_write 函数指针；src/file.c 中 struct file 通过 type 字段区分 FD_ENTRY/FD_DEVICE/FD_PIPE 并调用对应操作函数。
证据	[1] src/include/file.h · file struct file { enum { FD_NONE, FD_PIPE, FD_ENTRY, FD_DEVICE } type; ... struct pipe pipe; struct dirent ep; uint64 off; short major; }; [2] src/file.c · fileinput uint64 fileinput(struct file f, int user, uint64 addr, int n, uint64 off) { switch (f->type) { case FD_PIPE: return piperead(...); case FD_DEVICE: return (devsw + f->major)->read(...); case FD_ENTRY: return eread(f->ep, user, addr, off, n); } } [3] src/fat32.c · eread uint64 eread(struct dirent ep, int user, uint64 addr, uint64 off, uint64 n) - FAT32 文件读取实现	[1] src/fat32.c · struct fs struct fs{ uint devno; int valid; struct dirent image; struct Fat fat; struct entry_cache ecache; struct dirent root; void (disk_init)(struct dirent image); void (disk_read)(struct buf b, struct dirent image); void (disk_write)(struct buf b, struct dirent image); }; [2] src/file.c · struct file struct file { enum { FD_NONE, FD_PIPE, FD_ENTRY, FD_DEVICE } type; int ref; char readable; char writable; struct pipe pipe; struct dirent *ep; uint64 off; short major; };

Q05_002 · single_choice

具体文件系统后端 (Concrete File System Backend) 更接近哪种？

项目	方法A	方法B
答案	真实磁盘文件系统 (FAT32/Ext4/其他, 持久化存储)	混合挂载 (磁盘 FS + 内存 FS 均支持)
证据	[1] src/fat32.c · fat32_init int fat32_init(struct fs self_fs) - 读取 BPB (Boot Parameter Block), 解析 FAT32 卷结构, 初始化根目录簇 [2] src/diskio.c · disk_initialize DSTATUS disk_initialize(BYTE pdrv) { result = sd_init(spctrl, peripheral_input_khz, 0); return result == 0 ? RES_OK : RES_PARERR; } [3] src/bio.c · bread struct buf bread(uint dev, uint sectorno) { b = bget(dev, sectorno); if (!b->valid) { FatFs[dev].disk_read(b, FatFs[dev].image); b->valid = 1; } return b; }	[1] Makefile · FS_macro FS?=FAT ... CFLAGS += -D\$(FS) -D\$(MAC) [2] README.md · filesystem_options 文件系统选项: 基于 SD 卡 (make all fat=SD) / 基于内存 (make all fat=RAM) [3] src/disk.c · disk_init #ifdef RAM ramdisk_init(); #else disk_initialize(0); #endif

Q05_003 · short_answer

若支持 FAT32/Ext4: 它是自研还是第三方库/crate? (必须引用 Cargo.toml/Cargo.lock 或 Makefile 引入证据)

项目	方法A	方法B
答案	自研 FAT32 实现。src/fat32.c (1181 行) 完整实现 FAT32 文件系统, 包括 BPB 解析、簇分配 (alloc_clus)、FAT 表读写 (read_fat/write_fat)、目录项查找 (dirlookup)、文件读写 (eread/ewrite)。diskio.c 适配 FatFs 接口但仅作为 glue layer, 实际 FS 逻辑在 fat32.c 自研。证据: src/fat32.c:1-1181 完整 FAT32 实现; src/diskio.c:1-162 仅做磁盘接口适配。	自研实现。仓库为 C 语言项目 (无 Cargo.toml), FAT32 实现在 src/fat32.c 中, 约 1500+ 行代码, 包含完整的 FAT32 解析、目录操作、文件读写等。Makefile 中直接编译 src/fat32.o。
证据	[1] src/fat32.c · fat32_implementation 1181 行完整 FAT32 实现, 包含 fat32_init、dirlookup、eread、ewrite、alloc_clus、free_clus、read_fat、write_fat 等核心函数 [2] src/diskio.c · diskio_glue 162 行磁盘接口适配层, 实现 disk_initialize/disk_read/disk_write 调用 sd.c 的 SD 卡驱动	[1] Makefile · OBJS OBJS += ... \$K/fat32.o ... [2] src/fat32.c · fat32_init <pre>int fat32_init(struct fs self_fs) { ... if (strncmp((char const)(b->data + 82), "FAT32", 5)) panic("not FAT32 volume"); ... }</pre> [3] README.md · work_description 调试 FAT32 文件系统, 为用户程序提供稳定的文件操作接口

Q05_004 · short_answer

文件打开路径: 文件打开入口 (sys_open 或等价) → VFS 层 → 具体 FS open。列出 3-6 个关键节点并给证据。

项目	方法A	方法B
答案	文件打开路径: 1. sys_openat (src/sysfile.c:39-145) - 系统调用入口, 解析参数 dirfd/path/flags/mode; 2. ename (src/fat32.c:1068-1072) - 路径解析, 调用 lookup_path 遍历目录树; 3. dirlookup (src/fat32.c:863-933) - 在目录中查找指定文件名的 dirent; 4. filealloc (src/file.c:43-56) - 分配全局 file 结构; 5. fdalloc (src/sysfile.c:14-27) - 在进程 ofile 表中分配 FD; 6. 设置 file.type=FD_ENTRY/FD_DEVICE 并返回 fd。若 O_CREATE 标志则调用 create (src/fat32.c:556-598) 创建新文件。	1. sys_openat (src/sysfile.c) → 2. ename (src/fat32.c, 路径解析) → 3. dirlookup/ealloc (src/fat32.c, 目录查找/创建) → 4. filealloc (src/file.c, 分配 file 结构) → 5. fdalloc (src/sysfile.c, 分配 fd) → 6. 返回 fd 给用户
证据	[1] src/sysfile.c · sys_openat <pre>uint64 sys_openat() { argfd(0,&dirfd,&dirf); argstr(1, path, ...); argint(2, &flags); argint(3, &mode); ep = ename(dp,path,&devno); if(flags & O_CREATE) ep = create(dp,path, T_FILE, flags); f = filealloc(); fd = fdalloc(f); f->type = FD_ENTRY/FD_DEVICE; return fd; }</pre> [2] src/fat32.c · ename <pre>struct dirent ename(struct dirent env,char path,int devno) { return lookup_path(env,path, 0, name, devno); }</pre> [3] src/file.c · filealloc <pre>struct file* filealloc(void) { acquire(&ftable.lock); for(f = ftable.file; f < ftable.file + NFILE; f++) { if(f->ref == 0) { f->ref = 1; release(&ftable.lock); return f; } } release(&ftable.lock); return NULL; }</pre>	[1] src/sysfile.c · sys_openat <pre>uint64 sys_openat() { ... if((ep = ename(dp,path,&devno)) == NULL) { if(flags & O_CREATE){ ep = create(dp,path, T_FILE, flags); } ... if((f = filealloc()) == NULL (fd = fdalloc(f)) < 0) ... }</pre> [2] src/fat32.c · ename <pre>struct dirent ename(struct dirent env,char path,int devno) { char name[FAT32_MAX_FILENAME + 1]; return lookup_path(env,path, 0, name, devno); }</pre> [3] src/file.c · filealloc <pre>struct file* filealloc(void) { acquire(&ftable.lock); for(f = ftable.file; f < ftable.file + NFILE; f++){ if(f->ref == 0){ f- >ref = 1; release(&ftable.lock); return f; } } ... }</pre>

Q05_005 · short_answer

文件描述符表 (File Descriptor Table, FD Table) 的实现形态是什么？（固定数组/Vec/BTreeMap 等；必须给结构体定义证据）

项目	方法A	方法B
答案	Per-Process 固定数组。每个进程 struct proc 包含 struct file **ofile 指针数组，通过 kmalloc 分配 NOFILE(101) 个 file 指针。fdalloc 线性扫描 ofile[0..NOFILEMAX(p)) 寻找空位。全局 file 表 ftable 是固定数组 struct file file[NFILE] (NFILE=101)。证据：src/include/proc.h:149 定义 struct file **ofile；src/proc.c:255-267 分配初始化 ofile；src/sysfile.c:14-27 fdalloc 实现线性扫描。	每进程独立的可变长度指针数组（通过 kmalloc 动态分配）。struct proc 中包含 struct file ofile 字段，在 allocproc() 中通过 kmalloc(NOFILEsizeof(struct file)) 分配。
证据	<pre>[1] src/include/proc.h · proc.ofile struct proc { ... struct file ofile; int exec_close; struct dirent cwd; ... } [2] src/proc.c · procinit p->ofile = kmalloc(NOFILEsizeof(struct file)); p->exec_close = kmalloc(NOFILEsizeof(int)); for(int fd = 0; fd < NOFILE; fd++) { p->ofile[fd] = 0; p->exec_close[fd] = 0; } [3] src/sysfile.c · fdalloc static int fdallocfrom(struct file f,int start) { for(fd = start; fd < NOFILEMAX(p); fd++) { if(p->ofile[fd] == 0) { p- >ofile[fd] = f; return fd; } } return -EMFILE; } [4] src/file.c · ftable struct { struct spinlock lock; struct file file[NFILE]; } ftable;</pre>	<pre>[1] src/proc.h · struct proc struct proc { ... struct file ofile; int exec_close; ... }; [2] src/proc.c · allocproc p->ofile = kmalloc(NOFILE sizeof(struct file)); p->exec_close = kmalloc(NOFILEsizeof(int)); ... for(int fd = 0; fd < NOFILE; fd++){ p->ofile[fd] = NULL; ... } [3] src/include/param.h · NOFILE #define NOFILE 101 // open files per process</pre>

Q05_007 · short_answer

若存在缓存：驱逐策略是什么（LRU/Clock/FIFO/无驱逐）？必须指出判断依据（字段/算法分支）证据。

项目	方法A	方法B
答案	LRU (Least Recently Used)。bcache 维护双向链表，head.next 指向最近使用的 buffer，head.prev 指向最久未使用的 buffer。bget 分配时从 head.prev 向前扫描找 refcnt==0 的 buffer（src/bio.c:88-97）。brelse 释放时将 buffer 移到 head.next 位置（src/bio.c:128-137），实现 LRU 驱逐。判断依据：src/bio.c:53-54 注释明确说明"Sorted by how recently the buffer was used. head.next is most recent, head.prev is least."。	LRU (Least Recently Used)。在 src/bio.c 的 bget() 中，从 bcache.head.prev（最少使用端）向前查找 refcnt==0 的缓冲区进行回收；brelse() 中将释放的缓冲区移到 bcache.head.next（最近使用端）。

项目	方法A	方法B
证据	[1] src/bio.c · bcache_comment // Linked list of all buffers, through prev/next. // Sorted by how recently the buffer was used. // head.next is most recent, head.prev is least. [2] src/bio.c · brelse void brelse(struct buf *b) { ... b->next->prev = b->prev; b->prev->next = b->next; b->next = bcache.head.next; b->prev = &bcache.head; bcache.head.next->prev = b; bcache.head.next = b; release(&bcache.lock); } [3] src/bio.c · bget for(b = bcache.head.prev; b != &bcache.head; b = b->prev) { if(b->refcnt == 0) { b->dev = dev; b->sectorno = sectorno; b->valid = 0; b->refcnt = 1; release(&bcache.lock); acquiresleep(&b->lock); return b; } }	[1] src/bio.c · bget // Recycle the least recently used (LRU) unused buffer. for(b = bcache.head.prev; b != &bcache.head; b = b->prev){ if(b->refcnt == 0) { b->dev = dev; b->sectorno = sectorno; ... return b; } } [2] src/bio.c · brelse void brelse(struct buf *b) { ... b->refcnt--; if (b->refcnt == 0) { b->next->prev = b->prev; b->prev->next = b->next; b->next = bcache.head.next; b->prev = &bcache.head; bcache.head.next->prev = b; bcache.head.next = b; } ... }

Q05_008 · tri_state_impl

是否实现页缓存 (Page Cache)或与 mmap/文件映射共享缓存页？（必须三态）

项目	方法A	方法B
答案	not_found	stub
证据	[1] src/mmap.c · page_cache_scan 在 mmap.c 检索 page_cache/页缓存相关标识符，未发现独立页缓存结构。do_mmap 直接调用 fileread 读取文件内容到内存，无共享缓存机制 [2] src/vma.c · vma_shared_check struct vma 定义中无 shared 字段，alloc_mmap_vma 未处理 MAP_SHARED 标志，仅存储 fd 和 f_off [3] src/file.c · file_read_cache fileread 直接调用 eread/piperead/devsw[].read，无页缓存层	[1] src/mmap.c · do_mmap uint64 do_mmap(uint64 start, uint64 len, int prot, int flags, int fd, off_t offset) { ... if(fd != -1) { mmap_sz = f->ep->file_size - offset; ... for(int i = 0; i < page_n; ++i) { ... fileread(f, va, PGSIZE); ... } } ... } [2] src/vm.c · page_cache_search 在 src/vm.c 和 src/mmap.c 中搜索 page_cache、pagecache、page_cache 等关键词，未发现独立的页缓存结构。mmap 直接通过 fileread 读取文件内容到内存，无共享页缓存机制。

Q05_010 · tri_state_impl

是否实现 poll/select/epoll（或等价事件机制）？（必须三态）

项目	方法A	方法B
答案	not_found	stub
证据	[1] src/sysfile.c · grep_sys_poll_select_epoll 在 sysfile.c 及全仓库检索 sys_poll/sys_select/sys_epoll 标识符，0 命中。未实现事件等待机制 [2] src/include/syscall.h · syscall_table_scan 检索系统调用表，未发现 poll/select/epoll 相关定义	[1] src/syspoll.c · sys_ppoll uint64 sys_ppoll(){ return 0; } [2] src/poll.c · poll_header #include "include/timer.h" ... // 文件内容几乎为空，仅包含头文件引用，无实际 poll 实现 [3] src/include/poll.h · struct pollfd struct pollfd { int32 fd; int16 events; int16 revents; }; // 结构体定义存在但无完整实现

Q05_014 · tri_state_impl

是否实现网络 socket（作为 VFS 文件对象）？（必须三态）

项目	方法A	方法B
答案	not_found	stub
证据	[1] src/include/file.h · fd_socket_check struct file 的 type 枚举仅含 FD_NONE/FD_PIPE/FD_ENTRY/FD_DEVICE, 无 FD_SOCKET 类型 [2] src/sysfile.c · grep_sys_socket 检索 sys_socket/sys_bind/sys_listen/sys_connect 等网络系统调用, 0 命中。README.md 声称"完成了对本地回环地址的 Socket 支持"但代码中未发现实现 [3] src/include/socket.h · socket_header 仅定义 struct socket_connection 和 socket_init 声明, 无具体实现代码	[1] src/include/socket.h · struct socket_connection struct socket_connection{ int IP; int sock_opt; uint64 sock_addr; int passive_socket; char temp[MAX_LENGTH_OF_SOCKET]; }; void socket_init(void); int add_socket(int IP,int op); [2] src/file.c · socket_fd_type 在 src/file.c 中 struct file 的 type 枚举只有 FD_NONE, FD_PIPE, FD_ENTRY, FD_DEVICE, 无 FD_SOCKET。搜索 socket 相关实现, 仅在头文件中有结构体定义, 无实际 socket 系统调用实现。 [3] README.md · socket_note 完成了对本地回环地址的 Socket 支持 // 但代码中未见完整 socket 实现

Q05_015 · tri_state_impl

是否实现伪文件系统（devfs/procfs/sysfs）？（必须三态；若 implemented 需说明实现形态）

项目	方法A	方法B
答案	not_found	implemented
证据	[1] src/dev.c · devfs_scan devinit 创建/dev 目录 和/dev/null、/dev/zero、/dev/console 设备节点, 但通过 creat 在 FAT32 中创建普通文件, 非独立 devfs。检索 devfs/procfs/sysfs 标识符, 0 命中 [2] src/fat32.c · pseudo_fs_check 无伪文件系统实现, 设备通过 devsw 注册表访问, 非 VFS 挂载	[1] src/dev.c · devinit int devinit() { devnum = 0; dev = create(NULL, "/dev", T_DIR, 0); ... memset(devsw, 0, NDEVsizeof(struct devsw)); allocdev("console", consoleread, consolewrite); allocdev("null", nullread, nullwrite); allocdev("zero", zeroread, zerowrite); ... } [2] src/dev.c · struct devsw struct devsw { char name[DEV_NAME_MAX+1]; struct spinlock lk; int (read)(int, uint64, int); int (*write)(int, uint64, int); }; extern struct devsw devsw[]; [3] src/sysfile.c · sys_openat if(ep==dev&&devno==1&&strncmp(path+pathlen-3,"dev",4)){ ... }else{ f->type = FD_DEVICE; f->major = devno; ... } // 设备文件作为 FD_DEVICE 类型处理

Q05_017 · single_choice

文件数据块分配方式 (File Allocation Method, Stallings Ch12) 更接近哪种？

项目	方法A	方法B
答案	FAT 表内嵌空闲链（FAT32 特有）	链式分配 (Chained/Linked Allocation): 块通过指针链接

项目	方法A	方法B
证据	[1] src/fat32.c · read_fat uint32 read_fat(struct fs * self_fs, uint32 cluster) - 读取 FAT 表项获取下一簇号, FAT32_EOC 表示文件结束 [2] src/fat32.c · write_fat void write_fat(struct fs * self_fs, uint32 cluster, uint32 next_clus) - 写入 FAT 表项建立簇链 [3] src/fat32.c · alloc_clus static uint32 alloc_clus(struct fs * self_fs, uint8 dev) - 扫描 FAT 表找值为 0 的表项分配新簇, 设置 FAT32_EOC 标记	[1] src/fat32.c · read_fat static uint32 read_fat(struct fs * self_fs, uint32 cluster) { ... uint32 fat_sec = fat_sec_of_clus(self_fs, cluster, 1); struct buf b = bread(self_fs->devno, fat_sec); uint32 next_clus = (uint32 *) (b->data + fat_offset_of_clus(self_fs, cluster)); ... return next_clus; } [2] src/fat32.c · reloc_clus while (clus_num > entry->clus_cnt) { int clus = read_fat(self_fs, entry->cur_clus); if (clus >= FAT32_EOC) { ... } entry->cur_clus = clus; entry->clus_cnt++; } // 通过 FAT 表链式遍历 cluster

Q05_023 · short_answer

驱动框架接口是什么？（Rust Driver trait / C driver ops / 注册表；必须引用接口定义证据）

项目	方法A	方法B
答案	C 语言设备注册表 devsw。src/include/dev.h:23-28 定义 struct devsw { char name[DEV_NAME_MAX+1]; int (read)(int, uint64, int); int (write)(int, uint64, int); struct spinlock lk; }; devinit 通过 allocdev 注册设备到 devsw 数组。设备操作通过 devsw[major].read/write 间接调用。证据： src/dev.c:54-63 allocdev 实现；src/file.c:158-170 FD_DEVICE 类型读写通过 devsw 分发。	C driver ops 表（函数指针结构体）。在 src/dev.c 中定义 struct devsw，包含 name、lock、read、write 函数指针。驱动通过 allocdev() 注册到 devsw 数组中。
证据	[1] src/include/dev.h · devsw struct devsw { char name[DEV_NAME_MAX+1]; int (read)(int user_dst,uint64 addr,int n); int (write)(int user_dst,uint64 addr,int n); struct spinlock lk; }; [2] src/dev.c · allocdev int allocdev(char name,int (devread)(int, uint64, int),int (devwrite)(int, uint64, int)) { strncpy(devsw[devnum].name,name,DEV_NAME_MAX+1); initlock(&devsw[devnum].lk,name); devsw[devnum].read = devread; devsw[devnum].write = devwrite; devnum++; return 0; } [3] src/file.c · fileread case FD_DEVICE: if(f->major < 0 f->major >= getdevnum() !devsw[f->major].read) return -1; struct devsw mydev = devsw + f->major; acquire(&mydev->lk); r = mydev->read(1, addr, n); release(&mydev->lk); break;	[1] src/dev.c · struct devsw struct devsw { char name[DEV_NAME_MAX+1]; struct spinlock lk; int (read)(int, uint64, int); int (write)(int, uint64, int); }; extern struct devsw devsw[]; [2] src/dev.c · allocdev int allocdev(char name,int (devread)(int, uint64, int),int (*devwrite)(int, uint64, int)){ if(devnum == NDEV){ return -1; } strncpy(devsw[devnum].name,name,DEV_NAME_MAX+1); devsw[devnum].read = devread; devsw[devnum].write = devwrite; devnum++; return 0; }

Q05_024 · short_answer

驱动注册与初始化顺序是什么？（init_drivers/probe/driver_manager 等；列出 3-6 个关键节点并给证据）

项目	方法A	方法B
答案	初始化顺序 (src/main.c:54-62) : 1. disk_init() - 初始化 SD 卡硬件; 2. fs_init() - 初始化 FAT32 文件系统; 3. devinit() - 注册 console/null/zero 设备到 devsw; 4. fileinit() - 初始化全局 file 表。devinit 中调用 allocdev 注册具体设备驱动函数。证据: src/main.c:54-62 main 函数初始化序列; src/dev.c:28-45 devinit 实现。	1. cpuinit() → 2. printfinit() → 3. kpmallocinit() → 4. kvminithart() → 5. trapinithart() → 6. procinit() → 7. binit() → 8. disk_init() → 9. fs_init() → 10. devinit() → 11. fileinit() → 12. userinit()
证据	[1] src/main.c · main disk_init(); fs_init(); logbufinit(); devinit(); fileinit(); - 设备驱动初始化顺序 [2] src/dev.c · devinit int devinit() { devnum = 0; dev = create(NULL, "/dev", T_DIR, 0); ... memset(devsw, 0, NDEVsizeof(struct devsw)); allocdev("console", consoleread, consolewrite); allocdev("null", nullread, nullwrite); allocdev("zero", zeroread, zerowrite); return 0; } [3] src/sd.c · sd_init int sd_init(spi_ctrl spi, unsigned int input_clk_khz, int slot) - SD 卡驱动初始化, 配置 SPI 控制器	[1] src/main.c · main void main(...) { ... cpuinit(); printfinit(); kpmallocinit(); kvminithart(); trapinithart(); procinit(); binit(); disk_init(); fs_init(); devinit(); fileinit(); userinit(); ... } [2] src/dev.c · devinit int devinit() { ... allocdev("console", consoleread, consolewrite); allocdev("null", nullread, nullwrite); allocdev("zero", zeroread, zerowrite); ... } [3] src/bio.c · binit void binit(void) { initlock(&bcache.lock, "bcache"); ... }

Q05_029 · short_answer

MMIO 地址来源是什么? (DTB 提供 / 常量硬编码 / 物理→虚拟转换; 必须给证据)

项目	方法A	方法B
答案	常量硬编码。 src/include/memlayout.h:69-70 定义 #define VIRTIO0 0x10001000 和 #define VIRTIO0_V (VIRTIO0 + VIRT_OFFSET)。 src/sd.c 直接使用 (spi_ctrl*) SPI2_CTRL_ADDR 访问 SPI 控制器。无 DTB 解析, MMIO 地址在编译期确定。证据: src/include/memlayout.h 定义 MMIO 常量; src/sd.c:53-60 使用硬编码地址。	常量硬编码。所有设备 MMIO 地址在 src/include/memlayout.h 中定义为宏常量 (如 UART0=0x10000000L, CLINT=0x02000000L, PLIC=0x0c000000L)。 kvminithart() 中直接将这常量映射到内核页表。
证据	[1] src/include/memlayout.h · VIRTIO0 #define VIRTIO0 0x10001000 #define VIRTIO0_V (VIRTIO0 + VIRT_OFFSET) [2] src/sd.c · sd_init sd_init(spi_ctrl spi, ...) - spi 参数由调用者传入硬编码的 SPI2_CTRL_ADDR [3] src/diskio.c · spictrl static spi_ctrl spictrl = (spi_ctrl*) SPI2_CTRL_ADDR; - 硬编码 SPI 控制器地址	[1] src/include/memlayout.h · mmio_addresses #define UART0 0x10000000L #define UART0_V (UART0 + VIRT_OFFSET) #define CLINT 0x02000000L #define CLINT_V (CLINT + VIRT_OFFSET) #define PLIC 0x0c000000L #define PLIC_V (PLIC + VIRT_OFFSET) [2] src/vm.c · kvminithart void kvminithart() { ... kvmmap(UART0_V, UART0, PGSIZE, PTE_R PTE_W); kvmmap(CLINT_V, CLINT, 0x10000, PTE_R PTE_W); kvmmap(PLIC_V, PLIC, 0x400000, PTE_R PTE_W); ... }

Q05_030 · short_answer

多平台适配是如何通过构建/条件编译选择驱动的? (features/Kconfig/Makefile 规则; 必须给证据)

项目	方法A	方法B
答案	条件编译宏。src/include/plic.h:42-47 使用 #ifdef QEMU 区分 QEMU 和 k210 平台的中断号定义。src/trap.c 使用 #ifndef QEMU 处理 SBI 特定逻辑。Makefile 中通过 CFLAGS += -DQEMU 或-DK210 控制平台选择。证据：src/include/plic.h:42-47 QEMU/k210 条件编译；src/trap.c:252-255 SBI 特定处理。	Makefile 条件编译 + C 预处理器宏。Makefile 中定义 FS?=FAT 和 MAC?=SIFIVE_U，通过 CFLAGS += -D\$(FS) -D\$(MAC) 传递宏定义。代码中使用 #ifdef QEMU、#ifdef SIFIVE_U、#ifdef RAM、#ifdef SD 进行条件编译。
证据	[1] src/include/plic.h · QEMU_macro #ifdef QEMU // QEMU #define UART0_IRQ 4 #define UART1_IRQ 5 #else // k210 #define UART0_IRQ 4 #define UART1_IRQ 5 #endif [2] src/trap.c · QEMU_check #ifndef QEMU w_sip(r_sip() & ~2); // clear pending bit sbi_set_mie(); #endif [3] Makefile · makefile_platform 通过 CFLAGS += -DQEMU 或-DK210 控制平台特定编译	[1] Makefile · platform_macros FS?=FAT MAC?=SIFIVE_U ... CFLAGS += -D\$(FS) -D\$(MAC) ... ifeq (\$(MAC),SIFIVE_U) DISK:=\$K/link_null.o endif ifeq \$(MAC),QEMU) DISK:=\$K/link_disk.o endif [2] src/disk.c · disk_init #ifdef RAM ramdisk_init(); #else disk_initialize(0); #endif [3] src/vm.c · kvm_init #ifdef RAM kvmmap(RAMDISK, RAMDISK, 0x5000000, PTE_R PTE_W); #endif #ifdef SD kvmmap(SPI2_CTRL_ADDR, SPI2_CTRL_ADDR_P, SPI2_CTRL_SIZE, PTE_R PTE_W); #endif

Q05_034 · single_choice

I/O 控制技术 (I/O Control Techniques, Stallings Ch11) 更接近哪种？

项目	方法A	方法B
答案	中断驱动 I/O (Interrupt-Driven I/O)：设备完成后发中断通知 CPU	混合（小传输用中断，大传输用 DMA）
证据	[1] src/trap.c · devintr int devintr(void) - 处理外部中断，虽然 PLIC 代码被注释，但通过 SBI 接收中断通知 [2] src/sd.c · sd_interrupt SD 卡驱动同步轮询设备状态（sd_cmd 中的 do-while 循环），但整体中断框架通过 devintr 处理 [3] src/trap.c · interrupt_comment / else if (DISK_IRQ == irq) { disk_intr(); } / - 注释表明设计为中断驱动，但实际实现为轮询	[1] src/trap.c · devintr int devintr(void) { ... else if (0x8000000000000005L == scause) { timer_tick(); return 2; } ... } // 定时器中断驱动 [2] src/sd.c · sd_dummy static inline uint8_t sd_dummy(spi_ctrl* spi) { return spi_txx(spi, 0xFF); } // SD 卡驱动使用 PIO 轮询 [3] src/disk.c · disk_intr void disk_intr(void) { #ifdef SD // dmac_intr(DMAC_CHANNEL0); #endif } // 中断处理存在但实现不完整

Q05_035 · tri_state_impl

是否实现 DMA (Direct Memory Access, Stallings Ch11) 传输路径？（必须三态；搜索 dma_alloc / dma_map / dma_buf / virtio 描述符环等；virtio 的描述符环也算 DMA 等价机制）

项目	方法A	方法B
答案	not_found	stub

项目	方法A	方法B
证据	[1] src/ · grep_dma 检索 dma_alloc/dma_map/dma_buf 标识符, 0 命中。无 DMA 相关实现 [2] src/sd.c · sd_dma_check sd_read_blocks/sd_write_blocks 通过 SPI 逐字节传输, 无 DMA 描述符或环缓冲机制 [3] src/include/virtio.h · virtio_desc virtio.h 定义 virtio 寄存器结构, 但无 virtio-blk/virtio-net 驱动实现, 无描述符环使用	[1] src/ · dma_search 搜索 dma_alloc、dma_map、dma_buf、dma_sync、virtio_ring、vring_desc 等关键词, 未发现完整 DMA 实现。 [2] src/sd.c · sd_read_blocks int sd_read_blocks(spi_ctrl spi, void dst, uint32_t src_lba, size_t size) { ... do { uint8_t x = sd_dummy(spi); *p++ = x; ... } while (--n > 0); } // 使用 PIO 逐字节读取, 无 DMA [3] src/include/virtio.h · virtio_header struct VRingDesc { uint64 addr; uint32 len; uint16 flags; uint16 next; }; // virtio 描述符结构存在但无完整实现 [4] src/disk.c · virtio_implementation Makefile 中 OBJS 无 virtio_disk.o, src/ 目录中无 virtio_disk.c 实现文件。virtio 相关仅头文件定义。

06_sync_ipc — 同步互斥与进程间通信

Q06_001 · short_answer

该内核提供了哪些同步原语？（SpinLock/Mutex/RwLock/Semaphore/Condvar/WaitQueue 等；列出类型定义证据）

项目	方法A	方法B
答案	已实现 SpinLock（自旋锁）和 SleepLock（睡眠锁），以及 WaitQueue（等待队列）机制。SpinLock 定义于 src/include/spinlock.h:7-13（struct spinlock），核心函数 acquire/release 实现于 src/spinlock.c:25-75。SleepLock 定义于 src/include/sleeplock.h:9-17（struct sleeplock，内部封装 spinlock），核心函数 acquiresleep/releasesleep 实现于 src/sleeplock.c:25-45。WaitQueue 通过全局数组 waitq_pool[WAITQ_NUM] 实现，定义于 src/proc.c:28-30，操作函数包括 allocwaitq/findwaitq/delwaitq/waitq_push/waitq_pop。未发现 RwLock、Semaphore、Condvar 的独立实现。	SpinLock: src/include/spinlock.h struct spinlock { uint locked; char name; struct cpu cpu; }; SleepLock: src/include/sleeplock.h struct sleeplock { uint locked; struct spinlock lk; char name; int pid; }; WaitQueue: src/include/queue.h typedef struct{ void chan; struct spinlock lk; struct list head; }queue; Pipe: src/include/pipe.h struct pipe { struct spinlock lock; char data[PIPESIZE]; uint nread; uint nwrite; int readopen; int writeopen; }; Condition Variable (via sleep/wakeup): src/proc.c sleep(void chan, struct spinlock lk) / wakeup(void *chan)

项目	方法A	方法B
证据	[1] src/include/spinlock.h · spinlock struct spinlock { uint locked; char name; struct cpu cpu; }; [2] src/include/sleeplock.h · sleeplock struct sleeplock { uint locked; struct spinlock lk; char *name; int pid; }; [3] src/proc.c · waitq_pool queue waitq_pool[WAITQ_NUM]; int waitq_valid[WAITQ_NUM];	[1] src/include/spinlock.h · spinlock struct spinlock { uint locked; char name; struct cpu cpu; }; [2] src/include/sleeplock.h · sleeplock struct sleeplock { uint locked; struct spinlock lk; char name; int pid; }; [3] src/include/queue.h · queue typedef struct { void chan; struct spinlock lk; struct list head; } queue; [4] src/include/pipe.h · pipe struct pipe { struct spinlock lock; char data[PIPESIZE]; uint nread; uint nwrite; int readopen; int writeopen; };

Q06_002 · single_choice

Mutex 更接近哪种实现？

项目	方法A	方法B
答案	自旋锁 (Spinlock, Busy-Waiting)	B. 阻塞锁 (Blocking Mutex, 进入等待队列并挂起)
证据	[1] src/spinlock.c · acquire while(__sync_lock_test_and_set(&lk->locked, 1) != 0) ; // 自旋等待	[1] src/sleeplock.c · acquiresleep void acquiresleep(struct sleeplock *lk) { acquire(&lk->lk); while (lk->locked) { sleep(lk, &lk->lk); } lk->locked = 1; release(&lk->lk); }

Q06_004 · fill_in

sleep / wakeup 不变量 (Sleep-Wakeup Invariant) 分析，按格式填写： - sleep 入口函数: **(路径)** - **入睡前持有的锁:** (无则写 none) - 防丢 wakeup (Lost Wakeup Prevention) 机制: **(如: 持队列锁检查条件 / 无防护)** - **wakeup 函数:** (路径) - 唤醒与锁释放顺序: __ (先唤醒后释放 / 先释放后唤醒 / 其他)

项目	方法A	方法B
答案	sleep 入口函数: src/proc.c:542 (sleep 函数) 入睡前持有的锁: 调用者传入的 lk (通过参数), sleep 内部会先获取 p->lock 再释放 lk 防丢 wakeup (Lost Wakeup Prevention) 机制: 持 p->lock 检查条件并入队, wakeup 也持 waitq_pool_lk 保证原子性 wakeup 函数: src/proc.c:581 (wakeup 函数) 唤醒与锁释放顺序: 先唤醒 (将进程状态改为 RUNNABLE 并加入 readyq) 后释放锁 (wakeup 不直接持 lk, 由调用者负责)	sleep 入口函数: src/proc.c sleep(void chan, struct spinlock lk) 入睡前持有的锁: p->lock (进程锁) 防丢 wakeup (Lost Wakeup Prevention) 机制: 持 p->lock 检查条件并调用 sleep, wakeup 时持有 waitq lock wakeup 函数: src/proc.c wakeup(void *chan) 唤醒与锁释放顺序: 先唤醒后释放 (wakeup 中先修改状态为 RUNNABLE 并加入就绪队列, 然后 delwaitq 释放等待队列)
证据	[1] src/proc.c · sleep <pre>if(lk != &p->lock){ acquire(&p->lock); release(lk); } ... waitq_push(q,p); p->state = SLEEPING; sched();</pre> [2] src/proc.c · wakeup <pre>while((p = waitq_pop(q))!=NULL) { p->state = RUNNABLE; readyq_push(p); } delwaitq(q);</pre>	[1] src/proc.c · sleep <pre>void sleep(void chan, struct spinlock lk) { struct proc p = myproc(); if(lk != &p->lock){ acquire(&p->lock); release(lk); } queue q = findwaitq(chan); if(!q) q = allocwaitq(chan); waitq_push(q,p); p->state = SLEEPING; sched(); ... if(lk != &p->lock){ release(&p->lock); acquire(lk); } }</pre> [2] src/proc.c · wakeup <pre>void wakeup(void chan) { queue q = findwaitq(chan); if(q){ struct proc* p; while((p = waitq_pop(q))!=NULL){ p->state = RUNNABLE; readyq_push(p); } delwaitq(q); } }</pre>

Q06_009 · tri_state_impl

是否实现 futex? (必须三态)

项目	方法A	方法B
答案	stub	not_found
证据	[1] src/include/proc.h · FUTEX_WAIT src/include/proc.h:18-50 定义了 FUTEX_WAIT/WAKE/FD 等操作码 (0-13) [2] src/include/proc.h · do_futex src/include/proc.h:199 声明 int do_futex(int uaddr, int futex_op, int val, ktime_t timeout, int *addr2, int val2, int val3); 但 grep 搜索 do_futex 在 .c 文件中无实现 [3] syscall/syscall.c · grep_sys_futex 搜索 sys_futex 在 .c 文件中 0 命中; syscall 分发表中未见 futex 注册	[1] doc/内核实现--Futex.md · futex_doc 存在 futex 设计文档, 描述了 FUTEX_WAIT/FUTEX_WAKE/FUTEX_REQUEUE 操作, 但实际 syscall 未实现。 [2] syscall/sys.sh · syscall_generation 在 syscall/sys.sh 生成的系统调用列表中, 未见 futex 系统调用 entry。src/include/proc.h 中有 FUTEX_* 常量定义和 do_futex 声明, 但未在 syscall 表中注册。

Q06_011 · short_answer

若实现 signal handler: 用户态 handler 上下文如何构建? 是否存在 sigreturn 恢复原 trap frame? (必须给证据)

项目	方法A	方法B
答案	用户态 handler 上下文构建: 在 sighandle() 函数 (src/signal.c:124-200) 中, 分配新的 trapframe (tf = allocpage()), 设置 tf->epc 指向信号处理函数 (SIG_TRAMPOLINE + sig_handler 偏移), tf->a0=signum, tf->a1=default_sigaction, 然后将 p->trapframe 替换为新 tf。原 trapframe 保存在 sig_frame->tf 中。sigreturn 恢复机制: 存在 sigreturn() 函数 (src/signal.c:254-272), 从 p->sig_frame 链表弹出 frame, 恢复 p->trapframe = frame->tf, 然后释放 frame。调用路径: sys_rt_sigreturn (src/sys_sig.c:24-26) → sigreturn()。	用户态 handler 上下文构建: 在 src/signal.c sighandle() 中, 分配新的 trapframe 页, 设置 epc 指向 sig_trampoline 中的 sig_handler, 保存原 trapframe 到 sig_frame->tf。sigreturn 恢复: 存在, 在 src/signal.c sigreturn() 中, 从 sig_frame->tf 恢复原 trapframe, 并释放 sig_frame。
证据	[1] src/signal.c · sighandle <pre>frame->tf = p->trapframe; tf->epc = (uint64)(SIG_TRAMPOLINE + ((uint64)sig_handler - (uint64)sig_trampoline)); p->trapframe = tf;</pre> [2] src/signal.c · sigreturn <pre>freepage(p->trapframe); p->trapframe = frame->tf; p->sig_frame = frame->next; freepage(frame);</pre> [3] src/sys_sig.c · sys_rt_sigreturn <pre>uint64 sys_rt_sigreturn(void){ sigreturn(); return 0; }</pre>	[1] src/signal.c · sighandle <pre>void sighandle(void) { ... frame = allocpage(); tf = allocpage(); frame->tf = p->trapframe; tf->epc = (uint64)(SIG_TRAMPOLINE + ((uint64)sig_handler - (uint64)sig_trampoline)); ... p->trapframe = tf; frame->next = p->sig_frame; p->sig_frame = frame; }</pre> [2] src/signal.c · sigreturn <pre>void sigreturn(void) { ... if (NULL == p->sig_frame) { exit(-1); } struct sig_frame *frame = p->sig_frame; freepage(p->trapframe); p->trapframe = frame->tf; p->sig_frame = frame->next; freepage(frame); }</pre>

Q06_013 · single_choice

底层原子操作来源更接近哪种？

项目	方法A	方法B
答案	自定义汇编 (ldxr/stxr、lock xchg 等)	D. 未发现/不确定
证据	[1] src/spinlock.c · acquire <pre>while(__sync_lock_test_and_set(&lk->locked, 1) != 0) ; // GCC 内置原子, RISC-V 生成 amoswap.w.aq</pre> [2] src/spinlock.c · release <pre>__sync_lock_release(&lk->locked); // GCC 内置原子, RISC-V 生成 amoswap.w</pre> [3] src/spinlock.c · atomic_comment <pre>// On RISC-V, sync_lock_test_and_set turns into an atomic swap: amoswap.w a5, a5, (s1)</pre>	[1] src/spinlock.c · acquire <pre>while(__sync_lock_test_and_set(&lk->locked, 1) != 0) ;</pre> [2] src/spinlock.c · release <pre>__sync_lock_release(&lk->locked);</pre> [3] src/include/riscv.h · atomic_search 未发现自定义汇编原子操作 (如 ldxr/stxr 或 lock xchg)。使用 GCC builtin __sync_lock_test_and_set, 编译为 RISC-V amoswap 指令。非 Rust 标准库, 非自定义汇编。

Q06_014 · short_answer

死锁四必要条件 (Coffman Conditions) 在该内核中是否均成立？请逐条作答 (互斥 Mutual Exclusion / 持有并等待 Hold-and-Wait / 不可剥夺 No Preemption / 循环等待 Circular Wait), 并结合 SpinLock/Mutex 的实现给出证据或写「不适用」。

项目	方法A	方法B
答案	1. 互斥 (Mutual Exclusion): 成立。SpinLock 通过原子操作 __sync_lock_test_and_set 保证同一时刻只有一个 CPU 能获取锁 (src/spinlock.c:35) 。2. 持有并等待 (Hold-and-Wait): 成立。代码中存在嵌套锁场景, 如 acquiresleep 先持 lk->lk 再调用 sleep (src/sleeplock.c:27-32) ; fat32.c 中 elock 嵌套调用常见。3. 不可剥夺 (No Preemption): 成立。SpinLock 只能由持有者主动 release, 无强制剥夺机制 (src/spinlock.c:48-75) 。4. 循环等待 (Circular Wait): 可能成立。代码注释明确提到死锁风险 (src/proc.c:615-649) , 如 findchild/reparent 中为避免死锁而特殊处理锁顺序。	1. 互斥 (Mutual Exclusion): 成立。spinlock 通过 __sync_lock_test_and_set 保证同一时刻只有一个 CPU 持有锁。证据: src/spinlock.c acquire() 使用原子交换。2. 持有并等待 (Hold-and-Wait): 成立。acquiresleep() 持有 lk->lk 的同时调用 sleep() 等待 lk->locked 释放。证据: src/sleeplock.c acquiresleep()。3. 不可剥夺 (No Preemption): 成立。锁只能由持有者自愿 release, 内核不会强制剥夺。证据: src/spinlock.c release() 需调用者主动调用。4. 循环等待 (Circular Wait): 可能成立。若代码中多处获取多个锁且顺序不一致, 可能形成循环等待。内核未强制全局锁顺序。
证据	[1] src/spinlock.c · acquire while(__sync_lock_test_and_set(&lk->locked, 1) != 0) ; // 互斥 [2] src/sleeplock.c · acquiresleep acquire(&lk->lk); while(lk->locked){ sleep(lk, &lk->lk); } // 持有锁并等待 [3] src/proc.c · deadlock_comment // acquiring the lock first would cause a deadlock, since np might be an ancestor, and we already hold p->lock.	[1] src/spinlock.c · acquire void acquire(struct spinlock lk) { push_off(); if(holding(lk)) panic("acquire"); while(__sync_lock_test_and_set(&lk->locked, 1) != 0) ; ... } [2] src/sleeplock.c · acquiresleep void acquiresleep(struct sleeplock lk) { acquire(&lk->lk); while (lk->locked) { sleep(lk, &lk->lk); } lk->locked = 1; release(&lk->lk); } [3] src/spinlock.c · release void release(struct spinlock *lk) { if(!holding(lk)) panic("release"); ... __sync_lock_release(&lk->locked); pop_off(); }

Q06_015 · single_choice

内核对死锁 (Deadlock) 的处理策略更接近哪种?

项目	方法A	方法B
答案	死锁预防 (Deadlock Prevention): 通过锁顺序等消除 Coffman 必要条件	D. 忽略 (Ostrich Algorithm): 不处理, 依赖外部重启
证据	[1] src/proc.c · deadlock_prevention src/proc.c:615-649 注释说明为避免死锁而调整锁获取顺序, 如 exit() 中先释放锁再调用 wakeup [2] src/sysfile.c · deadlock_avoidance src/sysfile.c:357/460 注释提到 'Will this lead to deadlock?' 和 'elock dp may cause deadlock', 显示开发者意识到并尝试预防	[1] src/proc.c · deadlock_search 检索全部源码, 未发现死锁检测、避免或预防机制。无锁顺序规范注释, 无 Banker 算法, 无死锁恢复代码。 [2] src/spinlock.c · deadlock_prevention spinlock.c 中无死锁预防逻辑, 仅通过 holding() 检查防止同一 CPU 重复获取同一锁。

Q06_016 · tri_state_impl

是否存在全局锁顺序 (Lock Ordering) 规范或注释, 以预防嵌套锁导致的循环等待死锁 (Circular Wait Deadlock)? (必须三态; 若 implemented 需给出锁排序规则或 ABBA 锁检测代码证据)

项目	方法A	方法B
答案	stub	not_found

项目	方法A	方法B
证据	[1] <code>src/proc.c · lock_order_comment</code> <code>src/proc.c:615-649</code> 有多处注释说明特定场景下锁顺序会导致死锁，但这是局部注释，非全局规范 [2] <code>src/sysfile.c · deadlock_warning</code> <code>src/sysfile.c:357/460</code> 有注释警告潜在死锁，但无系统性锁排序规则或 ABBA 检测代码	[1] <code>src/proc.c · lock_ordering_search</code> 检索全部源码，未发现全局锁顺序规范、注释或 ABBA 死锁检测代码。 锁获取顺序依赖程序员约定，无强制规则。 [2] <code>src/sleeplock.c · lock_ordering_search</code> <code>sleeplock.c</code> 中无锁顺序说明。<code>acquiresleep</code> 先获取 <code>lk->lk</code> 再等待 <code>lk->locked</code>，但无全局顺序规范。

Q06_017 · tri_state_impl

是否实现管程/条件变量 (Monitor / Condition Variable, Stallings Ch5)? (必须三态; 搜索 Condvar / condition_variable / monitor / wait/notify/signal 等; 若 implemented 需区分 Hoare 语义 (等待者立即恢复) vs Mesa 语义 (等待者重新竞争锁))

项目	方法A	方法B
答案	not_found	implemented
证据	[1] <code>src/include · grep_condvar</code> 全仓库搜索 <code>Condvar condition_variable cond_wait cond_signal, 0</code> 命中: 无独立条件变量实现 [2] <code>src/proc.c · grep_monitor</code> 搜索 <code>monitor</code> 关键字, 0 命中; <code>sleep/wakeup</code> 是底层机制, 非管程抽象	[1] <code>src/proc.c · sleep</code> <code>void sleep(void chan, struct spinlock lk) { ... waitq_push(q,p); p->state = SLEEPING; sched(); ... } // 调用 sleep 后需重新获取锁并检查条件</code> [2] <code>src/proc.c · wakeup</code> <code>void wakeup(void chan) { queue q = findwaitq(chan); if(q){ struct proc* p; while((p = waitq_pop(q))!=NULL){ p->state = RUNNABLE; readyq_push(p); } delwaitq(q); } } // 唤醒后进程进入就绪队列, 需重新竞争锁</code> [3] <code>src/sleeplock.c · acquiresleep</code> <code>while (lk->locked) { sleep(lk, &lk->lk); } // 唤醒后需重新检查 lk->locked 条件, 符合 Mesa 语义</code>

Q06_018 · short_answer

经典同步问题验证 (Classic Synchronization Problems, Stallings Ch5): 以下三个经典问题在该内核中是否有对应实现或测试? - 生产者-消费者 (Producer-Consumer / Bounded Buffer): **(implemented/notfound + 证据)** - 读者-写者 (Readers-Writers): **_** (实现了读者优先/写者优先/公平? + 证据) - 哲学家就餐 (Dining Philosophers): **__** (implemented/not_found)

项目	方法A	方法B
答案	生产者 - 消费者 (Producer-Consumer / Bounded Buffer): not_found (pipe 实现本质是生产者 - 消费者模式, 但无独立测试或示例代码; grep 搜索 producer/consumer/bounded.buffer 0 命中) 读者 - 写者 (Readers-Writers): not_found (无 RwLock 实现, grep 搜索 reader/writer 0 命中, 仅 <code>errno.h</code> 中有 <code>ENOTTY</code> 误匹配) 哲学家就餐 (Dining Philosophers): not_found (grep 搜索 <code>dining.philosopher</code> 0 命中)	生产者 - 消费者 (Producer-Consumer / Bounded Buffer): implemented - <code>src/pipe.c</code> 实现管道, 使用环形缓冲区和 <code>sleep/wakeup</code> 同步, 典型生产者 - 消费者模式。证据: <code>piperead/pipewrite</code> 使用 <code>sleep</code> 等待缓冲区非空/非满。 读者 - 写者 (Readers-Writers): not_found - 未实现读写锁, 无读者优先/写者优先/公平策略实现。 哲学家就餐 (Dining Philosophers): not_found - 未找到相关实现或测试代码。

项目	方法A	方法B
证据	[1] src/pipe.c · pipe_producer_consumer pipe 的 pipewrite/piperead 实现生产者 - 消费者语义, 但无独立测试代码 [2] src/include · grep_classic_problems 搜索 producer consumer reader writer dining.philosopher, 仅 errno.h 误匹配, 无实现或测试	[1] src/pipe.c · piperead int piperead(struct pipe pi, int user, uint64 addr, int n) { ... while(pi->nread == pi->nwrite && pi->writeopen){ sleep(&pi->nread, &pi->lock); } ... } // 消费者等待生产者 [2] src/pipe.c · pipewrite int pipewrite(struct pipe pi, int user, uint64 addr, int n) { ... while(pi->nwrite == pi->nread + PIPESIZE){ sleep(&pi->nwrite, &pi->lock); } ... } // 生产者等待消费者 [3] src/include/spinlock.h · rwlock_search 未找到读写锁实现, 仅 spinlock 和 sleeplock。

07_security — 安全机制与权限模型

Q07_004 · short_answer

若存在权限检查：入口点与核心检查函数链路是什么？（列 2-5 个节点并给证据）

项目	方法A	方法B
答案	权限检查链路不完整，仅发现以下路径： 1. sys_faccessat (src/sysfile.c:493) → ename (查找文件) → 检查 F_OK → 返回 0（无实际权限比较） 2. sys_openat (src/sysfile.c:45) → 根据 flags 设置 f->readable/f->writable → 无 uid/gid 检查 3. ekstat (src/fat32.c:1038) → st_uid/st_gid 硬编码为 0, st_mode 硬编码为 0777 未发现 check_perm/inode_permission 等核心权限检查函数（grep 搜索 0 命中）	未发现完整的权限检查链路。仅有文件属性检查：sys_openat() (src/sysfile.c) -> ename() (src/fat32.c) -> ewrite() 检查 ATTR_READ_ONLY (src/fat32.c)。无 UID/GID 权限验证。
证据	[1] src/sysfile.c:493-565 · sys_faccessat sys_faccessat 实现：检查文件存在性后直接返回 0, 未比较进程 uid 与文件 st_uid [2] src/fat32.c:1038-1058 · ekstat ekstat 设置文件属性: st->st_uid = 0; st->st_gid = 0; st->st_mode = 0x1fff;	[1] src/sysfile.c · sys_openat uint64 sys_openat() - syscall entry point for open, calls ename() and create() but no permission check based on credentials [2] src/fat32.c · ewrite if ((entry->attribute & ATTR_READ_ONLY)) { return -1; } - Only checks read-only attribute, not owner permissions [3] src/trap.c · syscall void syscall(void) { num = p->trapframe->a7; ... syscallsnum; } - syscall dispatch without permission validation

Q07_009 · short_answer

本项目支持哪些架构（riscv64/aarch64/x86_64/loongarch64 等）？每种架构的安全相关初始化（特权级配置、PMP/MPU/SMEP 等）是否有代码证据？（必须逐架构作答，无证据写「未发现」）

项目	方法A	方法B
答案	仅支持 riscv64 架构。 riscv64 安全相关初始化证据： 1. 特权级配置：src/trap.c:156-158 usertrapret() 清除 SSTATUS_SPP 位切换到 U-mode 2. 中断使能：src/trap.c:55 w_sstatus(r_sstatus() SSTATUS_SIE) 开启 Supervisor 中断 3. 页表隔离：src/vm.c 使用 walkaddr/walkaddr1 进行地址翻译，但未见 PMP/MPU 配置代码 4. 未发现 PMP (Physical Memory Protection) 初始化代码 (grep PMP 仅 1 命中且无关) aarch64/x86_64/loongarch64：未发现支持代码 (仓库仅 src/sifive/ 目录含 RISC-V 特定代码)	仅支持 riscv64 架构。证据：1) Makefile 中 TOOLPREFIX=riscv64-linux-gnu-，QEMU=qemu-system-riscv64；2) src/include/riscv.h 定义 Sv39 页表 (SATP_SV39)；3) src/vm.c 中 kvminit() 配置内核页表权限 (PTE_R PTE_W PTE_X)；4) src/trap.c 中 trapinithart() 配置 stvec/sie 寄存器。未发现 PMP 配置代码，使用 S-mode/U-mode 分离作为特权级隔离机制。
证据	[1] src/trap.c:156-158 · usertrapret RISC-V S-mode 特权级切换: x &= ~SSTATUS_SPP; w_sstatus(x); [2] src/sifive/ · sifive_platform 仅包含 RISC-V SiFive 平台相关代码 (encoding.h/platform.h 等)，无其他架构目录 [3] Makefile · makefile_arch_check Makefile 中 TARGET=riscv64-unknown-elf-gcc，仅支持 RISC-V 64	[1] Makefile · TOOLPREFIX TOOLPREFIX=riscv64-linux-gnu- 和 QEMU = qemu-system-riscv64 - 明确指定 RISC-V 64 位架构 [2] src/include/riscv.h · SATP_SV39 #define SATP_SV39 (8L << 60) - 使用 Sv39 页表格式，三级页表 [3] src/vm.c · kvminit kvmmap(KERNBASE, KERNBASE, (uint64)etext - KERNBASE, PTE_R PTE_X) - 内核代码区映射为只读可执行 [4] src/trap.c · trapinithart w_stvec((uint64)kernelvec); w_sstatus(r_sstatus() SSTATUS_SIE); - 配置中断向量和启用中断

Q07_011 · tri_state_impl

是否实现了内核/用户页表隔离 (Kernel/User Page Table Isolation, KPTI 或等价机制)? (x86: CR3 KPTI / SMEP / SMAP; RISC-V: PMP / S-mode 分离; AArch64: TTBR0/TTBR1 隔离; 必须三态; 无则写未发现并列出已搜关键字)

项目	方法A	方法B
答案	not_found	stub

项目	方法A	方法B
证据	[1] <code>src/vm.c</code> · <code>grep_kpti</code> 搜索 <code>KPTI/SMEP/SMAP/PMP</code> 等关键词, 0 命中 [2] <code>src/proc.c:312-360</code> · <code>proc_pagetable</code> <code>proc_pagetable</code> 调用 <code>kvmcreate()</code> 创建页表, 内核和用户共享同一页表, 无隔离机制 [3] <code>src/include/memlayout.h:39</code> · <code>memlayout_scan</code> <code>memlayout.h</code> 注释提及 "for use by the kernel and user pages", 但未实现隔离	[1] <code>src/proc.c</code> · <code>proc_pagetable</code> <code>pagetable = kvmcreate(); ... memmove(pagetable, kernel_pagetable, PGSIZE);</code> - 用户页表复制内核页表, 非完全隔离 [2] <code>src/vm.c</code> · <code>kvmcreate</code> <code>pagetable_t kvmcreate() { ... memmove(pagetable, kernel_pagetable, PGSIZE); return pagetable; }</code> - 用户页表包含内核映射 [3] <code>src/include/riscv.h</code> · <code>PTE_U</code> <code>#define PTE_U (1L << 4) // 1 -> user can access</code> - 使用 <code>PTE_U</code> 位控制用户访问, 但内核映射对用户可见 [4] <code>src/trap.c</code> · <code>kpti_search</code> 检索 <code>KPTI/CR3_switch/page_table_isolation</code> 等关键字未发现。RISC-V 使用 <code>satp</code> 寄存器切换页表, 但用户页表包含内核映射 (<code>trampoline</code> 页), 非严格 <code>KPTI</code>。

Q07_012 · short_answer

UID/GID 字段是否在 `syscall` 路径上真实执行权限检查? (搜索 `check_perm/inode_permission`; 若只有字段无检查链须标注「仅有定义但未强制执行 ♦」; 给检查链证据或写「字段存在但无检查链」)

项目	方法A	方法B
答案	仅有定义但未强制执行 ♦ 证据: <code>src/include/proc.h:136-137</code> 定义 <code>uid/gid</code> 字段 <code>src/proc.c:236-237</code> 初始化为 0: <code>p->uid = 0; p->gid = 0</code> <code>src/sysproc.c:48-77</code> <code>sys_getuid/sys_setuid</code> 仅读写字段, 无权限验证 <code>src/fat32.c:1050-1051</code> <code>ekstat</code> 硬编码 <code>st_uid/st_gid</code> 为 0 <code>grep</code> 搜索 <code>check_perm/inode_permission/permission_check</code> 全仓库 0 命中 结论: <code>uid/gid</code> 字段存在但从未在 <code>open/exec/write</code> 等系统调用中用于权限检查, 所有进程默认 <code>uid=0</code> (<code>root</code> 权限), 无用户隔离	字段存在但无检查链 ♦
证据	[1] <code>src/include/proc.h:136-137</code> · <code>proc::uid/gid</code> <code>struct proc</code> 定义 <code>uid/gid</code> 字段 [2] <code>src/sysproc.c:48-77</code> · <code>sys_getuid/sys_setuid</code> <code>sys_getuid</code> 直接返回 <code>myproc()->uid</code>; <code>sys_setuid</code> 直接设置 <code>uid</code>, 无权限检查 [3] <code>src/fat32.c:1050-1051</code> · <code>ekstat</code> <code>st->st_uid = 0; st->st_gid = 0</code>; 所有文件所有者硬编码为 0	[1] <code>src/include/proc.h</code> · <code>proc</code> <code>struct proc { ... int uid; int gid; ... }</code> - <code>UID/GID</code> 字段定义存在 [2] <code>src/sysproc.c</code> · <code>sys_getuid</code> <code>uint64 sys_getuid(void) { return myproc()->uid; }</code> - 可读取 <code>UID</code> 但仅用于返回, 非权限检查 [3] <code>src/fat32.c</code> · <code>uid_permission_check</code> 在附件源码中检索 <code>uid/gid/permission/check_perm/inode_permission</code> 等关键字, <code>ewrite()</code> 仅检查 <code>ATTR_READ_ONLY</code> 属性, 无 <code>UID/GID</code> 所有权限验证。 <code>kstat</code> 结构中 <code>st_uid/st_gid</code> 字段存在但仅用于 <code>stat()</code> 返回, 非访问控制。 [4] <code>src/sysfile.c</code> · <code>sys_openat</code> <code>sys_openat()</code> 打开文件时仅检查 <code>O_RDONLY/O_WRONLY</code> 标志, 未验证进程 <code>UID/GID</code> 与文件所有者的匹配关系。

Q07_013 · single_choice

访问控制模型 (Access Control Model, Stallings Ch15) 更接近哪种?

项目	方法A	方法B
答案	D. 仅有特权级隔离 (ring0/ring3) , 无细粒度访问控制	A. 自主访问控制 DAC (Discretionary Access Control): 所有者自主设置权限 (Unix 权限位)
证据	[1] src/fat32.c:1058 · ekstat st->st_mode = 0x1ff; 所有文件权限位固定为 0777 (rwxrwxrwx), 无实际访问控制 [2] src/sysfile.c:504-565 · sys_faccessat sys_faccessat 未比较进程 uid/gid 与文件 st_uid/st_gid, 仅检查文件存在性 [3] src/proc.c:236-237 · allocproc 所有进程 uid=0, 无用户/组区分	[1] src/include/fat32.h · ATTR_READ_ONLY #define ATTR_READ_ONLY 0x01 #define ATTR_DIRECTORY 0x10 - FAT32 文件属性位, 类似 Unix 权限位但简化 [2] src/fat32.c · ewrite if ((Entry->attribute & ATTR_READ_ONLY)) { return -1; } - 基于文件属性的访问控制, 所有者可设置只读属性 [3] src/include/stat.h · kstat struct kstat { ... uint32 st_uid; uint32 st_gid; uint32 st_mode; ... } - 存在 UID/GID/mode 字段但仅用于 stat() 返回, 未用于访问控制决策

08_network — 网络子系统与协议栈

Q08_001 · tri_state_impl

是否存在网络子系统实现 (协议栈或 socket 层)? (必须三态)

项目	方法A	方法B
答案	stub	not_found
证据	[1] src/include/socket.h · socket_connection 定义了 struct socket_connection 结构体及 socket_init/add_socket 函数声明, 但在全仓库搜索 socket_init 和 add_socket 函数实现均为 0 命中, 仅有接口声明无实现代码 [2] src/include/defs.h · socket_function_scan 在 defs.h 中检索所有函数声明, 未发现 socket 相关函数实现; grep 搜索 socket_init add_socket 返回 0 匹配	[1] src/include/socket.h · socket.h 仅存在 socket.h 头文件定义 socket_connection 结构体, 但未见实际实现文件。声明了 socket_init() 和 add_socket() 但无对应实现。 [2] syscall/sys.sh · syscall_list 系统调用列表中未见 socket/bind/connect/sendto/recvfrom 等网络相关系统调用。最高 syscall 号为 276 (renameat2), 无网络 syscall。 [3] src/sysfile.c · sys_sendfile 存在 sys_sendfile 但这是文件间数据传输, 非网络 socket 发送。文件类型仅支持 FD_NONE/FD_PIPE/FD_ENTRY/FD_DEVICE, 无 FD_SOCKET。

Q08_004 · short_answer

选择一个发送路径 (优先 sys_sendto) , 追踪: syscall → 协议栈 → 网卡驱动。列 3-6 个关键节点并给证据。

项目	方法A	方法B
答案	无法追踪：未发现 socket 系统调用实现。sys_sendto 等 socket 相关系统调用在代码库中不存在（grep 搜索 sys_socket sys_sendto 返回 0 命中）。struct file 不支持 FD_SOCKET 类型，devsw 设备表仅注册 console/null/zero 设备，无网络相关设备。	未发现网络发送路径实现。代码中不存在 sys_sendto 或类似网络发送系统调用，无协议栈和网卡驱动实现。
证据	[1] src/sysfile.c · sendto_syscall_scan 检索 sys_sendto sys_socket sys_bind 等标识符，0 命中；sysfile.c 中仅实现文件/管道/设备相关 syscall，无网络 syscall [2] src/dev.c · devinit devinit() 仅调用 allocdev 注册 console/null/zero 三种设备，无网络设备安装；devsw[NDEV] 设备表无 socket 相关条目 [3] src/include/file.h · file_type_enum enum { FD_NONE, FD_PIPE, FD_ENTRY, FD_DEVICE } 无 FD_SOCKET 分支，filewrite/fileread 无法处理 socket 类型	[1] syscall/sys.sh · sendto_search 检索 syscall/sys.sh 中所有 entry 定义，未找到 sendto/sendmsg/socket 等网络发送相关系统调用。 [2] src/sysfile.c · sys_sendfile 存在 sys_sendfile 但这是文件描述符间数据传输（fin->fout），非网络发送。实现为 filesend() 函数，在文件层操作。 [3] src/ · network_driver 检索 virtio- net/e1000/network/ethernet 等关键词，未发现网卡驱动实现。仅存在 disk.c/virtio_disk.c 等存储驱动。

09_debug_error — 调试机制与错误处理

Q09_003 · short_answer

panic 路径会输出哪些诊断？（寄存器 dump/栈回溯/停机；必须引用实现证据）

项目	方法A	方法B
答案	panic 路径输出：(1) panic 消息字符串 (src/printf.c:141-143)；(2) 栈回溯 (backtrace) 打印返回地址序列 (src/printf.c:145)；(3) 停机 (for(;;) 死循环, src/printf.c:147-148)。trap 路径中 usertrap/kerneltrap 还会打印 scause/sepc/stval 及 trapframe 寄存器 dump (src/trap.c:115-117, 190-195)，调用 trapframedump() 输出 a0-a7/t0-t6/s0-s11/sp/gp/tp/epc/ra 等寄存器 (src/trap.c:250-277)。	panic 路径输出：1) "panic: "前缀消息 2) 用户提供的错误字符串 3) 换行 4) 栈回溯 (backtrace) 5) 进入无限循环停机。不直接输出寄存器 dump，但 trapframedump() 可用于 trapframe 寄存器转储。
证据	[1] src/printf.c · panic panic(): printf("panic: "); printf(s); printf("\n"); backtrace(); panicked = 1; for(;;) ; [2] src/trap.c · trapframedump trapframedump() 实现于 src/trap.c:250-277，打印 trapframe 中所有通用寄存器 (a0-a7, t0-t6, s0-s11, sp, gp, tp, epc, ra)。 [3] src/trap.c · usertrap usertrap 中意外异常处理: printf("\nusertrap(): unexpected scause %p pid=%d %s\n", r_scause(), p->pid, p->name); printf("sepc=%p stval=%p\n", r_sepc(), r_stval()); trapframedump(p->trapframe); (src/trap.c:115-117)	[1] src/printf.c · panic void panic(char s) { printf("panic: "); printf(s); printf("\n"); backtrace(); panicked = 1; for(;;) ; } [2] src/printf.c · backtrace void backtrace() { uint64 fp = (uint64)r_fp(); uint64 bottom = (uint64)PGROUNDUP((uint64)fp); printf("backtrace:\n"); while (fp < bottom) { uint64 ra = (fp - 1); printf("%p\n", ra - 4); fp = (uint64)(fp - 2); } } [3] src/trap.c · trapframedump void trapframedump(struct trapframe *tf) - prints all registers (a0-a7, t0-t6, s0-s11, ra, sp, gp, tp, epc) from trapframe.

Q09_007 · short_answer

错误码/错误类型体系是什么？（errno/Result/Error enum；给类型定义与传播点证据）

项目	方法A	方法B
答案	C 语言 errno 宏定义体系。定义于 src/include/errno.h:1-107，包含 98+ 个错误码宏 (EPERM/ENOENT/ESRCH/EINTR/EIO/ENOMEM/EACCES/EFAULT/EINVAL/ENOSYS 等)。系统调用通过 return -1 或 return -ERRNO 传播错误，如 sysproc.c 中多个 syscall 返回 -1 表示失败 (src/sysproc.c:17,21,78,90)。文件操作返回 -1 表示错误 (src/file.c:218,221,246 等)。无 Rust 风格 Result/Error enum。	Unix errno 风格错误码体系：使用正整数错误码定义于 src/include/errno.h，系统调用失败时返回 -errno（负值）。无 Result/Error enum 类型。
证据	[1] src/include/errno.h · errno_definitions src/include/errno.h 定义标准 errno 宏：#define EPERM 1, #define ENOENT 2, #define ENOSYS 38, #define EINVAL 22, #define EFAULT 14 等共 98+ 个错误码。 [2] src/sysproc.c · sys_exit 系统调用通过 return -1 传播错误，如 src/sysproc.c:17: if(copyin(...)) != 0) return -1; 多个 syscall 使用此模式。 [3] src/file.c · filealloc 文件操作返回 -1 表示错误：src/file.c:218-221: if(...) return -1; 无统一 errno 设置机制，调用者需自行判断错误类型。	[1] src/include/errno.h · errno_definitions #define EPERM 1, #define ENOENT 2, #define ESRCH 3, #define EINTR 4, #define EIO 5, ... #define EMFILE 24, #define ENOSYS 38 - standard Unix errno values. [2] src/sysfile.c · sys_openat Returns -1 on error, specific errno like -EMFILE for file descriptor exhaustion: if((f = filealloc()) == NULL (fd = fdalloc(f)) < 0) return -EMFILE; [3] src/sysproc.c · sys_clone Returns -1 on failure: if((np = allocproc(p, 1)) == NULL) return -1; - error propagates as negative return value.

Q09_008 · tri_state_impl

是否存在 trace/perf/ftrace 等跟踪机制或 tracepoints？（必须三态）

项目	方法A	方法B
答案	not_found	stub

项目	方法A	方法B
证据	[1] <code>src/ · grep_trace_perf_scan</code> 搜索 <code>'trace perf ftrace tracepoint'</code>, 13 个命中均为无关内容: <code>src/printf.c</code> 的 <code>backtrace</code> 调用 (4 次), <code>src/include/proc.h</code> 的 <code>CLONE_PTRACE</code> 标志 (3 次, 仅定义未实现 <code>ptrace</code>), <code>src/sifive/encoding.h</code> 的 <code>MCONTROL_ACTION_TRACE_*</code> (3 次, RISC-V 调试寄存器定义), <code>syscall/syscall.c:11</code> 的注释 <code>'// trace'</code>。未发现 <code>tracepoints</code> 插入或 <code>perf/ftrace</code> 实现。 [2] <code>src/include/proc.h · CLONE_PTRACE</code> <code>src/include/proc.h:68,78</code> 定义 <code>CLONE_PTRACE/CLONE_UNTRACED</code> 标志, 但 <code>grep</code> 未发现 <code>ptrace</code> 系统调用实现, 仅为预留标志位。 [3] <code>syscall/syscall.c · trace_comment</code> <code>syscall/syscall.c:11</code> 仅有注释 <code>'// trace'</code>, 无实际 <code>trace</code> 逻辑。未发现 <code>tracepoints</code> 插入到关键路径 (如 <code>syscall/trap/sched</code>)。	[1] <code>src/include/proc.h · tmask</code> <code>struct proc { ... int tmask; ... }; - trace mask field in process structure for per-process syscall tracing.</code> [2] <code>syscall/syscall.c · syscall</code> <code>if ((p->tmask & (1 << num)) != 0) { printf("pid %d: %s -> %d\n", p->pid, sysnames[num], p->trapframe->a0); } - basic syscall tracing when trace mask bit is set.</code> [3] <code>src/ · tracepoint/ftrace/perf</code> <code>Searched for tracepoint, ftrace, perf, trace_buffer symbols - no comprehensive tracing infrastructure found. Only basic syscall trace mask exists.</code>